

GOLANG LEARNING PATH

Belajar Golang dari Nol

Panduan lengkap belajar Go: dari pemula hingga mahir

Level Pemula (18 topik) • Level Menengah (11 topik)
Level Mahir (7 topik) • Fullstack Project

Daftar Isi

Pendahuluan

Bab 1 — Level Pemula

Bab 2 — Level Menengah

Bab 3 — Level Mahir

Bab 4 — Fullstack Project

Panduan lengkap belajar Go (Golang) dari pemula hingga mahir.

📁 Struktur Tutorial

```
learning-path-go/
├── exercises/           # Level PEMULA (18 topik)
│   ├── 01-hello.go      - Hello World
│   ├── 02-variabel.go   - Variabel & Tipe Data
│   ├── 03-fungsi.go     - Fungsi
│   ├── 04-slice.go      - Array & Slice
│   ├── 05-struct.go     - Struct
│   ├── 06-struct-method.go - Struct & Method
│   ├── 07-interface.go  - Interface
│   ├── 08-polymorphism.go - Polymorphism
│   ├── 09-goroutine.go  - Goroutine & Channel
│   ├── 10-error.go      - Error Handling
│   ├── 11-testing.go    - Testing
│   ├── 12-api.go        - REST API sederhana
│   ├── 13-pointer.go    - Pointer
│   ├── 14-map.go        - Map
│   ├── 15-defer-panic-recover.go - Defer/Panic/Recover
│   ├── 16-loop-switch.go - Loop & Switch
│   ├── 17-string.go     - Manipulasi String
│   └── 18-module/       - Go Modules
├── middle/             # Level MENENGAH (11 topik)
│   ├── context/         - Context (timeout, cancel, value)
│   ├── json/            - JSON encoding/decoding
│   ├── file-io/         - File & directory operations
│   ├── concurrency/     - WaitGroup, Mutex, Select
│   ├── testing/         - Table-driven test, benchmark
│   ├── database/        - SQLite CRUD (go get modernc.org/sqlite)
│   ├── logging/         - Standard log, multi-writer
│   ├── middleware/      - HTTP middleware pattern
│   ├── env/             - Environment variables config
│   ├── auth/            - JWT authentication
│   └── docker/          - Docker multi-stage build
├── expert/             # Level MAHIR (7 topik)
│   ├── grpc/            - gRPC & Protocol Buffers
│   ├── kubernetes/      - K8s deployment & HPA
│   ├── cicd/            - GitHub Actions pipeline
│   ├── profiling/       - Pprof, race detector, patterns
│   ├── design-patterns/ - Repository, Factory, Builder, Singleton
│   ├── websocket/       - Real-time WebSocket (go get gorilla/websocket)
│   └── graphql/         - GraphQL API (go get graphql-go/graphql)
└── fullstack/          # FULLSTACK PROJECT
    ├── api/             - Go REST API (CRUD + JSON file storage)
    └── frontend/        - React frontend (CDN atau Vite)
```

🔗 Cara Belajar

1. Level Pemula

```
# Jalankan setiap file secara berurutan
cd exercises
go run 01-hello.go
go run 02-variabel.go
# ... lanjutkan sampai 18-module
```

2. Level Menengah

```
cd middle
# Topik 1: Context
cd context && go run main.go
# Topik 2: JSON
cd ../json && go run main.go
# ... dan seterusnya
```

3. Level Mahir

```
cd expert
# Desain Patterns
cd design-patterns && go run main.go
# WebSocket (install dulu: go get github.com/gorilla/websocket)
cd ../websocket && go run main.go
# GraphQL
cd ../graphql && go run main.go
```

4. Fullstack Project

```
cd fullstack
```

```
# Terminal 1: Backend
cd api && go run main.go
```

```
# Terminal 2: Frontend (buka di browser)
cd frontend
# Buka index.html langsung di browser
# Atau jalankan: python3 -m http.server 3000
```

🔗 Quick Reference

Commands Dasar

```
go run main.go           # Jalankan Go program
go build -o app .         # Build binary
go test ./... -v -cover   # Test dengan coverage
go mod init example.com/app # Init module
go get github.com/package # Install dependency
go mod tidy               # Bersihkan dependencies
go fmt ./...              # Format kode
```

Debug & Profiling

```
go run -race main.go      # Race detector
go test -bench=. -benchmem # Benchmark
go tool pprof cpu.out      # Profiling
```

Build untuk Production

```
GOOS=linux GOARCH=amd64 go build -o app .
# Output: binary Linux 64-bit (static)
```

💡 Tips Belajar

1. **Praktek, bukan hanya baca:** Jalankan setiap contoh
2. **Modifikasi kode:** Ubah nilai, tambah fungsi baru
3. **Cari tahu error:** Error adalah guru terbaik
4. **Baca dokumentasi:** go.dev/doc
5. **Konsisten:** Sedikit setiap hari lebih baik dari banyak sekali

📖 Referensi

- [Go by Example](#) - Belajar dari contoh
- [Effective Go](#) - Best practices
- [Go Wiki](#) - Komunitas
- [Go Playground](#) - Coba Go di browser

🐳 Docker

```
cd middle/docker
docker build -t go-app .
docker run -p 8080:8080 go-app
docker compose up -d      # Dengan database
```

📄 License

MIT - Belajar, gunakan, dan bagikan dengan bebas.

Bab 1 — Level Pemula

Level pemula terdiri dari 18 topik dasar Go, dari Hello World hingga Go Modules.

exercises/01-hello.go

```
// File: 01-hello.go
// Level: Beginner
// Topik: Hello World
//
// Ini adalah program pertama di Go.
// Setiap program Go dimulai dari package main dan fungsi main().

package main

import "fmt"

func main() {
    // fmt.Println() digunakan untuk mencetak teks ke layar
    // String diapit oleh tanda petik dua ""
    fmt.Println("Selamat belajar Go!")
    fmt.Println("Halo malam!")
}
```

exercises/02-variabel.go

```
// File: 02-variabel.go
// Level: Beginner
// Topik: Variabel dan Tipe Data
//
// Go memiliki beberapa cara untuk mendeklarasikan variabel:
// 1. var nama tipe = nilai -> deklarasi eksplisit
// 2. nama := nilai -> deklarasi singkat (type inference)
// 3. const nama = nilai -> konstanta (tidak bisa diubah)

package main

import "fmt"

func main() {
    // Deklarasi eksplisit dengan var
    var nama string = "Golang" // string: kumpulan karakter
    var umur int = 2           // int: bilangan bulat

    // Deklarasi singkat dengan :=
    // Go akan otomatis mendeteksi tipe data
    aktif := true // bool: true/false
    versi := 1.21 // float64: bilangan desimal

    fmt.Println("Bahasa:", nama)
    fmt.Println("Versi:", umur)
    fmt.Println("Sedang belajar:", aktif)

    // Konstanta - nilainya tidak bisa diubah setelah dideklarasikan
    const appName = "Go Learning"
    fmt.Println("Aplikasi:", appName)

    // Multiple variabel dalam satu baris
    var a, b int = 1, 2
    fmt.Println("a =", a, "b =", b)

    // Zero value: variabel tanpa nilai awal
    var defaultInt int // 0
    var defaultString string // "" (string kosong)
    var defaultBool bool // false
    fmt.Printf("Default: int=%d, string='%s', bool=%v\n",
        defaultInt, defaultString, defaultBool)
}
```

```
// File: 03-fungsi.go
// Level: Beginner
// Topik: Fungsi
//
// Fungsi adalah blok kode yang bisa dipanggil berulang kali.
// Syntax: func namaFungsi(parameter) (returnType) { body }

package main

import "fmt"

// Fungsi dengan parameter dan return value
// a dan b bertipe int, return value bertipe int
func tambah(a int, b int) int {
    return a + b
}

// Fungsi dengan satu parameter dan return string
func sapa(nama string) string {
    return "Halo, " + nama + "!"
}

// Fungsi dengan multiple return values
// Mengembalikan dua nilai: string dan int
func info() (string, int) {
    return "Golang", 2024
}

// Fungsi dengan named return values
// Variabel sudah dideklarasikan di signature fungsi
func bagi(a, b int) (hasil int, err bool) {
    if b == 0 {
        return 0, true // error: pembagi nol
    }
    hasil = a / b // tidak perlu := karena sudah dideklarasikan
    err = false
    return // naked return - mengembalikan hasil dan err
}

// Variadic function - menerima jumlah parameter tidak tetap
func jumlahkan(angka ...int) int {
    total := 0
    for _, n := range angka {
        total += n
    }
    return total
}

func main() {
    // Memanggil fungsi tambah
    hasil := tambah(5, 3)
    fmt.Println("5 + 3 =", hasil)

    // Memanggil fungsi sapa
    fmt.Println(sapa("Teman"))

    // Menangkap multiple return values
    bahasa, tahun := info()
    fmt.Println(bahasa, "dirilis", tahun)

    // Named return
    h, isErr := bagi(10, 2)
    fmt.Println("10/2 =", h, "Error:", isErr)

    // Variadic function
    total := jumlahkan(1, 2, 3, 4, 5)
    fmt.Println("Total:", total)

    // Anonymous function - fungsi tanpa nama
    sayHello := func(name string) string {
        return "Hello " + name
    }
    fmt.Println(sayHello("World"))
}
```

```
// File: 04-slice.go
// Level: Beginner
// Topik: Array dan Slice
//
// Array: kumpulan data dengan ukuran tetap
// Slice: array dinamis (ukuran bisa berubah)

package main

import "fmt"

func main() {
    // ARRAY - ukuran tetap [5] berarti 5 elemen
    var angka [5]int = [5]int{1, 2, 3, 4, 5}
    fmt.Println("Array:", angka)
    fmt.Println("Elemen ke-2:", angka[1]) // Index dimulai dari 0

    // SLICE - dinamis, tidak perlu ukuran
    var fruits []string // Deklarasi slice kosong
    fruits = []string{"apel", "jeruk", "mangga"}
    fruits = append(fruits, "pisang") // append untuk menambah elemen
    fmt.Println("Buah:", fruits)
    fmt.Println("Panjang:", len(fruits))
    fmt.Println("Kapasitas:", cap(fruits))

    // Slice dari array
    sliceDariArray := angka[1:4] // index 1 sampai 3 (4 tidak termasuk)
    fmt.Println("Slice dari array:", sliceDariArray)

    // Make - membuat slice dengan kapasitas awal
    numbers := make([]int, 3, 5) // len=3, cap=5
    numbers[0] = 10
    numbers[1] = 20
    numbers[2] = 30
    numbers = append(numbers, 40, 50)
    fmt.Println("Numbers:", numbers)

    // Loop over slice dengan range
    fmt.Println("\nDaftar Buah:")
    for i, fruit := range fruits {
        fmt.Printf("%d: %s\n", i, fruit)
    }

    // Loop tanpa index (gunakan _)
    fmt.Println("\nNama Buah:")
    for _, fruit := range fruits {
        fmt.Println("-", fruit)
    }

    // Slice 2 dimensi
    matrix := [][]int{
        {1, 2, 3},
        {4, 5, 6},
        {7, 8, 9},
    }
    fmt.Println("Matrix:", matrix)

    // Copy slice
    src := []int{1, 2, 3}
    dst := make([]int, len(src))
    copy(dst, src)
    fmt.Println("Copy:", dst)
}
```



```
// File: 05-struct.go
// Level: Beginner
// Topik: Struct
//
// Struct adalah kumpulan field (seperti class di OOP lain)
// Digunakan untuk membuat tipe data kustom

package main

import "fmt"

// Mendefinisikan struct Mahasiswa
// Field: Nama (string), Usia (int), NIM (string)
type Mahasiswa struct {
    Nama string
    Usia int
    NIM  string
}

// Method: fungsi yang menempel pada struct
// (m Mahasiswa) disebut receiver - mirip method di OOP
func (m Mahasiswa) Tampil() string {
    return fmt.Sprintf("%s (%s, %d th)", m.Nama, m.NIM, m.Usia)
}

// Method dengan pointer receiver
// Bisa mengubah nilai asli struct
func (m *Mahasiswa) Ultah() {
    m.Usia++ // menambah usia, perubahan terasa di luar fungsi
}

func main() {
    // Cara 1: inisialisasi dengan field names
    anggi := Mahasiswa{
        Nama: "Anggi",
        Usia: 20,
        NIM:  "12345",
    }

    // Cara 2: inisialisasi berdasarkan urutan field
    budi := Mahasiswa{"Budi", 22, "67890"}

    // Cara 3: deklarasi lalu isi
    var citra Mahasiswa
    citra.Nama = "Citra"
    citra.Usia = 19
    citra.NIM = "11111"

    fmt.Println("Anggi:", anggi.Tampil())
    fmt.Println("Budi:", budi.Tampil())
    fmt.Println("Citra:", citra.Tampil())

    // Method pointer receiver
    fmt.Println("\nSebelum ultah:", anggi.Usia)
    anggi.Ultah()
    fmt.Println("Setelah ultah:", anggi.Usia)

    // Struct embedding (seperti inheritance)
    type MahasiswaAktif struct {
        Mahasiswa // embedded struct
        Semester  int
        IPK        float64
    }

    aktif := MahasiswaAktif{
        Mahasiswa: Mahasiswa{"Dewi", 21, "22222"},
        Semester:  4,
        IPK:       3.5,
    }
    fmt.Printf("\nMahasiswa Aktif: %s, Semester %d, IPK %.2f\n",
        aktif.Nama, aktif.Semester, aktif.IPK)
}
```

```
// File: 06-struct-method.go
// Level: Beginner
// Topik: Struct dan Method (lanjutan)
//
// Method adalah fungsi yang receiver-nya adalah struct.
// Dua jenis receiver:
// 1. Value receiver (m Mobil) - tidak mengubah nilai asli
// 2. Pointer receiver (m *Mobil) - bisa mengubah nilai asli

package main

import "fmt"

// Mendefinisikan struct Mobil
type Mobil struct {
    Merek string
    Warna string
    Tahun int
}

// Value receiver - hanya membaca data
// Method ini tidak mengubah struct aslinya
func (m Mobil) TampilInfo() string {
    return fmt.Sprintf("Mobil %s berwarna %s tahun %d",
        m.Merek, m.Warna, m.Tahun)
}

// Pointer receiver - bisa mengubah data struct asli
// Mengubah tahun mobil menjadi 2025
func (m *Mobil) Perbaiki() {
    m.Tahun = 2025
}

// Value receiver - method dengan parameter tambahan
func (m Mobil) IsTua() bool {
    return m.Tahun < 2020
}

func main() {
    // Inisialisasi struct Mobil
    gojek := Mobil{Merek: "Gojek", Warna: "Hitam", Tahun: 2020}

    // Memanggil method value receiver
    fmt.Println(gojek.TampilInfo())

    // Memanggil method value receiver (cek kondisi)
    fmt.Println("Apakah mobil tua?", gojek.IsTua())

    // Memanggil method pointer receiver - mengubah data
    gojek.Perbaiki()
    fmt.Println("Setelah diperbaiki:", gojek.TampilInfo())

    // Method bisa dipanggil dari pointer atau value
    // Go otomatis mengkonversi
    mobilPtr := &gojek
    mobilPtr.Perbaiki()

    // Chain method tidak default di Go, tapi bisa dibuat
    // dengan mengembalikan pointer
    type Builder struct {
        parts []string
    }
    (builder)
}
```

```
// File: 07-interface.go
// Level: Beginner
// Topik: Interface
//
// Interface adalah kumpulan method signatures (kontrak).
// Tipe data yang memiliki semua method dalam interface
// secara otomatis dianggap mengimplementasi interface tersebut.
// Ini disebut "structural typing" atau "duck typing".

package main

import "fmt"

// Mendefinisikan interface dengan satu method GetInfo()
// Semua tipe yang memiliki method GetInfo() string otomatis
// mengimplementasi interface ini
type Info interface {
    GetInfo() string
}

// Struct Benda dengan field Nama dan Harga
type Benda struct {
    Nama    string
    Harga   int
}

// Struct Benda mengimplementasi interface Info
// karena memiliki method GetInfo() string
func (b Benda) GetInfo() string {
    return fmt.Sprintf("Benda: %s, Harga: Rp%d", b.Nama, b.Harga)
}

// Struct Hewan dengan field Nama dan Umur
type Hewan struct {
    Nama    string
    Umur    int
}

// Struct Hewan juga mengimplementasi interface Info
func (h Hewan) GetInfo() string {
    return fmt.Sprintf("Hewan: %s, Umur: %d tahun", h.Nama, h.Umur)
}

// Fungsi yang menerima interface Info
// Bisa menerima Benda, Hewan, atau tipe lain yang
// mengimplementasi interface Info
func TampilInfo(item Info) {
    // Memanggil method yang didefinisikan di interface
    fmt.Println(item.GetInfo())
}

// Type assertion: memeriksa tipe asli dari interface
func Jenis(item Info) {
    switch v := item.(type) {
    case Benda:
        fmt.Printf("Ini benda dengan harga Rp%d\n", v.Harga)
    case Hewan:
        fmt.Printf("Ini hewan umur %d tahun\n", v.Umur)
    default:
        fmt.Println("Tipe tidak dikenal")
    }
}

func main() {
    // Membuat instance Benda dan Hewan
    sepatu := Benda{Nama: "Sepatu", Harga: 500000}
    kucing := Hewan{Nama: "Kucing", Umur: 3}

    // Memanggil fungsi dengan parameter interface
    TampilInfo(sepatu)
    TampilInfo(kucing)

    // Type assertion
    Jenis(sepatu)
    Jenis(kucing)

    // Interface kosong (any) - bisa menampung tipe apapun
    var apapun any = "Hello"
    fmt.Println("any:", apapun)
    apapun = 42
    fmt.Println("any:", apapun)
    apapun = Benda{"Meja", 100000}
    fmt.Println("any:", apapun.(Benda).Nama)
}
```

```
// File: 08-polymorphism.go
// Level: Beginner
// Topik: Polymorphism (Polimorfisme) via Interface
//
// Polimorfisme = kemampuan objek memiliki banyak bentuk.
// Di Go, polimorfisme dicapai melalui interface.
// Satu interface bisa menampung berbagai tipe data selama
// tipe tersebut mengimplementasi semua method di interface.

package main

import (
    "fmt"
    "math"
)

// Interface BangunDatar - kontrak untuk semua bangun datar
// Setiap bangun datar harus memiliki method Luas() dan Nama()
type BangunDatar interface {
    Luas() float64 // method untuk menghitung luas
    Nama() string  // method untuk mengembalikan nama bangun
}

// Struct Persegi
type Persegi struct {
    Sisi float64
}

// Persegi mengimplementasi interface BangunDatar
func (p Persegi) Luas() float64 {
    return p.Sisi * p.Sisi // rumus luas persegi: sisi x sisi
}
func (p Persegi) Nama() string {
    return "Persegi"
}

// Struct Lingkaran
type Lingkaran struct {
    JariJari float64
}

// Lingkaran mengimplementasi interface BangunDatar
func (l Lingkaran) Luas() float64 {
    return math.Pi * l.JariJari * l.JariJari // rumus luas:  $\pi \times r^2$ 
}
func (l Lingkaran) Nama() string {
    return "Lingkaran"
}

// Struct Segitiga
type Segitiga struct {
    Alas    float64
    Tinggi float64
}

// Segitiga mengimplementasi interface BangunDatar
func (s Segitiga) Luas() float64 {
    return 0.5 * s.Alas * s.Tinggi // rumus luas:  $\frac{1}{2} \times a \times t$ 
}
func (s Segitiga) Nama() string {
    return "Segitiga"
}

// Fungsi yang menerima interface BangunDatar
// Bisa menerima Persegi, Lingkaran, Segitiga, dll
// Inilah polimorfisme: satu fungsi bisa menangani banyak tipe
func CetakLuas(bd BangunDatar) {
    fmt.Printf("%s memiliki luas %.2f\n", bd.Nama(), bd.Luas())
}

func main() {
    // Slice berisi berbagai tipe yang implementasi BangunDatar
    benda := []BangunDatar{
        Persegi{Sisi: 5},
        Lingkaran{JariJari: 7},
        Segitiga{Alas: 5, Tinggi: 10},
        Persegi{Sisi: 10},
        Lingkaran{JariJari: 3.5},
    }

    // Loop dan panggil CetakLuas() untuk setiap elemen
    // Meskipun tipenya berbeda, mereka bisa diperlakukan sama
    for _, item := range benda {
        CetakLuas(item)
    }
}
```

```
// File: 09-goroutine.go
// Level: Beginner
// Topik: Goroutine dan Channel
//
// Goroutine: thread ringan di Go, dijalankan dengan keyword "go"
// Channel: jembatan komunikasi antar goroutine
// - ch <- value : kirim data ke channel
// - <-ch       : terima data dari channel
// Paradigma: "Don't communicate by sharing memory, share memory by communicating"

package main

import (
    "fmt"
    "time"
)

// Fungsi yang akan dijalankan sebagai goroutine
// nama: identifier goroutine, ch: channel untuk mengirim pesan
func cetakPesan(nama string, ch chan string) {
    for i := 0; i < 3; i++ {
        time.Sleep(100 * time.Millisecond) // simulasi kerja
        ch <- fmt.Sprintf("%s: pesan %d", nama, i+1) // kirim ke channel
    }
}

func main() {
    // Membuat channel dengan make()
    // Channel bisa mengirim data string
    ch := make(chan string)

    // Menjalankan goroutine dengan keyword "go"
    // Dua goroutine berjalan secara concurrent
    go cetakPesan("Goroutine 1", ch)
    go cetakPesan("Goroutine 2", ch)

    // Menerima data dari channel sebanyak 6 kali
    // (3 dari goroutine 1 + 3 dari goroutine 2)
    for i := 0; i < 6; i++ {
        pesan := <-ch // blocking: menunggu sampai ada data
        fmt.Println(pesan)
    }

    // Contoh channel buffered
    buffered := make(chan int, 3) // buffer capacity 3
    buffered <- 1                  // tidak blocking karena buffer masih kosong
    buffered <- 2
    buffered <- 3
    // buffered <- 4 // ini akan blocking karena buffer penuh

    fmt.Println("Buffered channel:")
    fmt.Println(<-buffered)
    fmt.Println(<-buffered)
    fmt.Println(<-buffered)

    // Contoh: unbuffered channel akan blocking sampai ada receiver
    // go func() {
    //     ch2 := make(chan string)
    //     ch2 <- "test" // blocking jika tidak ada yang menerima
    // }()
}
```

```
// File: 10-error.go
// Level: Beginner
// Topik: Error Handling
//
// Go tidak menggunakan try-catch untuk error handling.
// Sebaliknya, fungsi mengembalikan error sebagai return value.
// Pola umum: if err != nil { handle error }
//
// error adalah interface built-in:
// type error interface {
//     Error() string
// }

package main

import (
    "errors" // package untuk membuat error
    "fmt"
    "strconv" // package untuk konversi string
)

// Fungsi bagi dengan error handling
// Mengembalikan error jika input tidak valid
func bagi(a, b string) (float64, error) {
    // strconv.ParseFloat mengembalikan (float64, error)
    angkaA, err := strconv.ParseFloat(a, 64)
    if err != nil {
        // errors.New() membuat error baru
        return 0, errors.New("angka pertama tidak valid: " + err.Error())
    }

    angkaB, err := strconv.ParseFloat(b, 64)
    if err != nil {
        return 0, errors.New("angka kedua tidak valid: " + err.Error())
    }

    if angkaB == 0 {
        // Error untuk kasus khusus: pembagian dengan nol
        return 0, errors.New("pembagi tidak boleh nol")
    }
}
```

```

    // Jika sukses, kembalikan hasil dan nil (tanpa error)
    return angkaA / angkaB, nil
}

// Custom error type - implementasi interface error
type ValidationError struct {
    Field string
    Value string
    Msg    string
}

// Method Error() membuat ValidationError mengimplementasi interface error
func (e *ValidationError) Error() string {
    return fmt.Sprintf("validasi gagal: %s '%s' - %s",
        e.Field, e.Value, e.Msg)
}

// Fungsi registrasi dengan custom error
func register(nama string, umur int) error {
    if nama == "" {
        return &ValidationError{
            Field: "nama",
            Value: nama,
            Msg:    "nama tidak boleh kosong",
        }
    }
    if umur < 17 {
        return &ValidationError{
            Field: "umur",
            Value: fmt.Sprintf("%d", umur),
            Msg:    "minimal umur 17 tahun",
        }
    }
    return nil
}

func main() {
    // Contoh 1: Basic error handling
    hasil, err := bagi("10", "2")
    if err != nil {
        fmt.Println("Error:", err)
    } else {
        fmt.Println("Hasil:", hasil)
    }

    // Contoh 2: Error karena pembagi nol
    _, err = bagi("10", "0")
    if err != nil {
        fmt.Println("Error:", err)
    }

    // Contoh 3: Custom error dengan type assertion
    err = register("", 20)
    if err != nil {
        // Type assertion untuk mengecek tipe error
        var valErr *ValidationError
        if errors.As(err, &valErr) {
            fmt.Printf("Validation Error: field=%s, msg=%s\n",
                valErr.Field, valErr.Msg)
        } else {
            fmt.Println("Error:", err)
        }
    }

    err = register("Anggi", 15)
    if err != nil {
        var valErr *ValidationError
        if errors.As(err, &valErr) {
            fmt.Printf("Validation Error: field=%s, msg=%s\n",
                valErr.Field, valErr.Msg)
        }
    }

    // Contoh 4: Panic (error fatal) dan Recover
    // defer func() {
    //     if r := recover(); r != nil {
    //         fmt.Println("Recovered:", r)
    //     }
    // }()
    // panic("Terjadi masalah fatal!")
}

```

```
// File: 11-testing.go
// Level: Beginner
// Topik: Unit Testing
//
// Go memiliki built-in testing package.
// Aturan:
// 1. File test harus berakhiran _test.go
// 2. Fungsi test harus dimulai dengan Test
// 3. Parameter fungsi test: t *testing.T
// 4. Jalankan dengan: go test

package main

import "testing"

// Fungsi yang akan di-test
func Tambah(a, b int) int {
    return a + b
}

// TestTambah adalah fungsi unit test
// Nama fungsi harus dimulai dengan "Test"
func TestTambah(t *testing.T) {
    hasil := Tambah(2, 3)
    expected := 5

    // Jika hasil tidak sesuai, laporkan error
    if hasil != expected {
        t.Errorf("Tambah(2, 3) = %d; harus %d", hasil, expected)
    }
}

// Test dengan multiple cases
func TestTambahMulti(t *testing.T) {
    type testCase struct {
        a, b      int
        expected int
    }

    cases := []testCase{
        {2, 3, 5},
        {-1, 1, 0},
        {0, 5, 5},
        {100, -50, 50},
    }

    for _, tc := range cases {
        hasil := Tambah(tc.a, tc.b)
        if hasil != tc.expected {
            t.Errorf("Tambah(%d, %d) = %d; harus %d",
                tc.a, tc.b, hasil, tc.expected)
        }
    }
}

// Test helper - untuk setup dan teardown
func setupTest(t *testing.T) func() {
    t.Log("Setup sebelum test")
    return func() {
        t.Log("Teardown setelah test")
    }
}

func TestDenganSetup(t *testing.T) {
    teardown := setupTest(t)
    defer teardown() // dipanggil setelah test selesai

    hasil := Tambah(1, 1)
    if hasil != 2 {
        t.Fail()
    }
}

// Cara menjalankan:
// go test -v          -> verbose output
// go test -run TestTambah -> jalankan test spesifik
// go test -cover       -> lihat code coverage
//
// NOTE: file ini harus di-rename menjadi *_test.go
// agar bisa dijalankan dengan `go test`
// Contoh: go test 11-testing.go 11-testing_test.go -v
```

```
// File: 12-api.go
// Level: Beginner
// Topik: REST API sederhana dengan net/http
//
// Go memiliki package net/http untuk membuat HTTP server.
// Tidak perlu framework tambahan untuk API sederhana.
// Package encoding/json untuk serialisasi JSON.

package main

import (
    "encoding/json" // encode/decode JSON
    "fmt"
    "log"           // logging
    "net/http"      // HTTP server & client
)

// Struct Pengguna dengan JSON tags
// Tags memberi tahu encoder/decoder JSON tentang nama field
type Pengguna struct {
    ID    int    `json:"id"` // di JSON jadi "id"
    Nama string `json:"nama"` // di JSON jadi "nama"
}

// Data dummy - slice of Pengguna
var pengguna = []Pengguna{
    {ID: 1, Nama: "Anggi"},
    {ID: 2, Nama: "Budi"},
}

// Handler untuk endpoint /pengguna
// http.ResponseWriter: untuk menulis response
// http.Request: data request dari client
func handlerPengguna(w http.ResponseWriter, r *http.Request) {
    // Set header Content-Type ke JSON
    w.Header().Set("Content-Type", "application/json")

    // Encode slice pengguna ke JSON dan kirim ke response
    err := json.NewEncoder(w).Encode(pengguna)
    if err != nil {
        // Jika error, kirim HTTP 500 Internal Server Error
        http.Error(w, err.Error(), http.StatusInternalServerError)
    }
}

func main() {
    // Mendaftarkan handler ke path /pengguna
    http.HandleFunc("/pengguna", handlerPengguna)

    // Handler untuk root path
    http.HandleFunc("/", func(w http.ResponseWriter, r *http.Request) {
        fmt.Fprintln(w, "Selamat datang di API Go!")
        fmt.Fprintln(w, "Endpoint: /pengguna")
    })

    // Menjalankan server di port 8080
    // log.Fatal akan mencatat error dan exit jika server gagal start
    fmt.Println("Server berjalan di http://localhost:8080")
    log.Fatal(http.ListenAndServe(":8080", nil))
}

// Untuk mengakses:
// curl http://localhost:8080/pengguna
// curl http://localhost:8080/
```



```
// File: 13-pointer.go
// Level: Beginner
// Topik: Pointer
//
// Pointer adalah variabel yang menyimpan alamat memori variabel lain.
// Alih-alih menyimpan nilai, pointer menyimpan lokasi di memori.
//
// Operator:
// & -> mengambil alamat memori (address-of)
// * -> mengambil nilai dari alamat (dereference)

package main

import "fmt"

func main() {
    // Variabel biasa
    var x int = 10
    fmt.Println("Nilai x:", x)
    fmt.Println("Alamat x:", &x) // &x = alamat memori x

    // Pointer p menunjuk ke alamat x
    // *int berarti "pointer to int"
    var p *int = &x
    fmt.Println("Nilai p (alamat x):", p)
    fmt.Println("Dereference p:", *p) // *p = nilai di alamat tersebut

    // Mengubah nilai melalui pointer
    *p = 20 // set nilai di alamat yang ditunjuk p menjadi 20
    fmt.Println("Setelah *p = 20, x menjadi:", x) // x ikut berubah

    // Pointer ke struct
    type User struct {
        Name string
        Age  int
    }

    // Membuat pointer langsung ke struct dengan &
    u := &User{Name: "Anggi", Age: 20}
    fmt.Println("User:", u.Name, u.Age) // Go otomatis dereference

    // Pointer sebagai parameter fungsi - pass by reference
    // Tanpa pointer, fungsi hanya menerima copy (pass by value)
    angka := 5
    double(angka) // pass by value - tidak mengubah asli
    fmt.Println("Setelah double:", angka) // masih 5
    doublePtr(&angka) // pass by reference - mengubah asli
    fmt.Println("Setelah doublePtr:", angka) // jadi 10

    // Nil pointer - pointer yang belum punya alamat
    var ptr *int
    if ptr == nil {
        fmt.Println("ptr adalah nil (belum指向 alamat manapun)")
    }

    // Zero value dari pointer adalah nil
    // Mengakses *ptr saat nil akan menyebabkan panic
}

// Pass by value - parameter adalah copy
func double(n int) {
    n = n * 2 // hanya mengubah copy lokal
}

// Pass by reference - parameter adalah pointer
func doublePtr(n *int) {
    *n = *n * 2 // mengubah nilai asli melalui pointer
}
```

```
// File: 14-map.go
// Level: Beginner
// Topik: Map
//
// Map adalah struktur data key-value (seperti dictionary/hash map).
// Syntax: map[KeyType]ValueType
// - Key: harus comparable (bisa dibandingkan dengan ==)
// - Value: tipe data apapun

package main

import "fmt"

func main() {
    // Deklarasi map literal
    // map[string]int berarti key string, value int
    scores := map[string]int{
        "Anggi": 90, // key: "Anggi", value: 90
        "Budi": 85,
    }
    fmt.Println("Scores:", scores)

    // Menambah dan mengupdate data
    scores["Citra"] = 95 // menambah key baru
    scores["Anggi"] = 100 // mengupdate key yang sudah ada
    fmt.Println("Updated:", scores)

    // Mengakses value dengan dua return value
    // val : nilai jika key ada
    // exists: true jika key ada, false jika tidak
    val, exists := scores["Deni"]
    if exists {
        fmt.Println("Deni:", val)
    } else {
        fmt.Println("Deni tidak ditemukan")
    }

    // Menghapus key dengan delete()
    delete(scores, "Budi")
    fmt.Println("Setelah delete Budi:", scores)

    // Iterasi map dengan range (urutan tidak dijamin)
    fmt.Println("\nDaftar scores:")
    for name, score := range scores {
        fmt.Printf("%s: %d\n", name, score)
    }

    // Map sebagai set (hanya butuh key, tanpa value berarti)
    visited := map[string]bool{}
    visited["/home"] = true
    visited["/about"] = true

    if visited["/home"] {
        fmt.Println("Home sudah dikunjungi")
    }

    // Map dengan nilai kompleks
    students := map[string]map[string]int{
        "Anggi": {"matematika": 90, "fisika": 85},
        "Budi": {"matematika": 80, "fisika": 95},
    }
    fmt.Println("\nNilai matematika Anggi:", students["Anggi"]["matematika"])

    // Nil map - harus diinisialisasi dengan make() sebelum digunakan
    var m map[string]int
    if m == nil {
        fmt.Println("m adalah nil map")
        m = make(map[string]int) // inisialisasi
        m["test"] = 1
        fmt.Println("Nilai m[\"test\"]:", m["test"])
    }

    // Panjang map
    fmt.Println("Jumlah students:", len(scores))
}
```

```
// File: 15-defer-panic-recover.go
// Level: Beginner
// Topik: Defer, Panic, Recover
//
// Defer : menjadwalkan eksekusi fungsi setelah fungsi induk selesai
//         (biasanya untuk cleanup: close file, close connection)
// Panic : menghentikan program secara paksa (seperti exception)
// Recover: menangkap panic agar program tidak crash
//
// Eksekusi defer menggunakan LIFO (Last In First Out)

package main

import "fmt"

func main() {
    // Defer: fungsi dijalankan saat fungsi main() selesai
    // Defer menggunakan stack (LIFO): yang terakhir didefer dijalankan pertama
    defer fmt.Println("1. Ini dijalankan terakhir (defer 1)")
    defer fmt.Println("2. Ini dijalankan kedua terakhir (defer 2)")
    fmt.Println("3. Ini dijalankan pertama")
    fmt.Println()

    // Defer untuk cleanup - pattern umum di Go
    readFile()

    // Panic & Recover - mencegah program crash
    fmt.Println("\n--- Demo safeDivision ---")
    safeDivision(10, 2) // sukses
    safeDivision(10, 0) // panic, tapi di-recover

    fmt.Println("Program tetap berjalan tanpa crash!")
    fmt.Println()

    // Contoh penggunaan: HTTP handler dengan recover
    handleRequest("valid")
    handleRequest("panic")
    fmt.Println("Server tetap berjalan setelah panic handler")
}

// readFile mensimulasikan pembacaan file
func readFile() {
    fmt.Println("Membuka file...")
    // defer akan selalu dieksekusi, bahkan jika terjadi panic
    defer fmt.Println("Menutup file...") // cleanup
    fmt.Println("Membaca konten file...")
    // File otomatis "tertutup" karena defer di atas
}

// safeDivision melakukan pembagian dengan proteksi panic
func safeDivision(a, b int) {
    // Recover hanya berguna di dalam deferred function
    defer func() {
        if r := recover(); r != nil {
            // recover() mengembalikan nilai panic
            // Program tidak jadi crash
            fmt.Println("Recover dari panic:", r)
        }
    }()

    if b == 0 {
        // panic menghentikan fungsi saat ini
        panic("Pembagian dengan nol!")
    }
    result := a / b
    fmt.Printf("%d / %d = %d\n", a, b, result)
}

// handleRequest mensimulasikan handler HTTP dengan recover
func handleRequest(req string) {
    defer func() {
        if r := recover(); r != nil {
            fmt.Printf("Request '%s' menyebabkan panic: %v\n", req, r)
            fmt.Println("Request ditangani tanpa crash server")
        }
    }()

    fmt.Printf("Memproses request: %s\n", req)
    if req == "panic" {
        panic("sesuatu error!")
    }
    fmt.Println("Request sukses")
}
```

```
// File: 16-loop-switch.go
// Level: Beginner
// Topik: Perulangan dan Percabangan
//
// Go hanya punya satu keyword untuk loop: "for"
// - for init; condition; post {}
// - for condition {} (seperti while)
// - for {} (infinite loop)
//
// Switch: percabangan multi-kondisi
// - switch value {}
// - switch {} (dengan kondisi)

package main

import "fmt"

func main() {
```

```

// 1. FOR BASIC
fmt.Println("=== For basic ===")
// init: i := 0, condition: i < 5, post: i++
for i := 0; i < 5; i++ {
    fmt.Print(i, " ")
}
fmt.Println()

// 2. FOR SEBAGAI WHILE
fmt.Println("\n=== For as while ===")
sum := 0
for sum < 10 { // hanya condition, tidak ada init/post
    sum += 3
    fmt.Print(sum, " ")
}
fmt.Println()

// 3. INFINITE LOOP + BREAK
fmt.Println("\n=== Infinite loop dengan break ===")
count := 0
for {
    count++
    if count > 3 {
        break // keluar dari loop
    }
    fmt.Print(count, " ")
}
fmt.Println()

// 4. CONTINUE - skip iterasi
fmt.Println("\n=== Continue ===")
for i := 0; i < 10; i++ {
    if i%2 == 0 {
        continue // skip angka genap
    }
    fmt.Print(i, " ")
}
fmt.Println()

// 5. RANGE OVER SLICE
fmt.Println("\n=== Range slice ===")
fruits := []string{"apel", "jeruk", "mangga"}
// i: index, fruit: value
for i, fruit := range fruits {
    fmt.Printf("%d: %s\n", i, fruit)
}

// Range tanpa index
fmt.Println("\nRange tanpa index:")
for _, fruit := range fruits {
    fmt.Println("-", fruit)
}

// Range over map
fmt.Println("\nRange over map:")
scores := map[string]int{"Anggi": 90, "Budi": 85}
for name, score := range scores {
    fmt.Printf("%s: %d\n", name, score)
}

// 6. SWITCH - percabangan nilai
fmt.Println("\n=== Switch value ===")
day := "senin"
switch day {
case "senin":
    fmt.Println("Hari kerja pertama")
case "sabtu", "minggu": // multiple case
    fmt.Println("Akhir pekan")
default:
    fmt.Println("Hari biasa")
}

// 7. SWITCH DENGAN KONDISI
fmt.Println("\n=== Switch condition ===")
score := 85
switch {
case score >= 90:
    fmt.Println("A")
case score >= 80:
    fmt.Println("B")
case score >= 70:
    fmt.Println("C")
default:
    fmt.Println("D")
}

// 8. SWITCH TYPE - untuk interface
fmt.Println("\n=== Switch type ===")
var value any = 42
switch v := value.(type) {
case int:
    fmt.Println("Integer:", v)
case string:
    fmt.Println("String:", v)
default:
    fmt.Println("Tipe lain:", v)
}

```

```

}
```

```
// File: 17-string.go
// Level: Beginner
// Topik: Manipulasi String
//
// String di Go adalah immutable (tidak bisa diubah).
// Manipulasi string menggunakan package "strings" dan "strconv".
// Untuk operasi string yang banyak, gunakan strings.Builder.

package main

import (
    "fmt"
    "strconv" // konversi string <-> number
    "strings" // manipulasi string
)

func main() {
    // 1. OPERASI DASAR STRING
    s := "Hello, Golang!"
    fmt.Println("String asli:", s)
    fmt.Println("Panjang:", len(s)) // jumlah byte
    fmt.Println("Char pertama:", string(s[0])) // H
    fmt.Println("Char ke-7:", string(s[7])) // G
    fmt.Println("Substring [0:5]:", s[0:5]) // Hello
    fmt.Println("Substring [7:]:", s[7:]) // Golang!

    // 2. PACKAGE STRINGS
    fmt.Println("\n=== Package strings ===")
    fmt.Println("ToUpper:", strings.ToUpper(s))
    fmt.Println("ToLower:", strings.ToLower(s))
    fmt.Println("Contains 'Go':", strings.Contains(s, "Go"))
    fmt.Println("HasPrefix 'Hello':", strings.HasPrefix(s, "Hello"))
    fmt.Println("HasSuffix '!' :", strings.HasSuffix(s, "!"))

    // Replace
    fmt.Println("Replace Go->Rust:", strings.Replace(s, "Go", "Rust", 1))
    fmt.Println("ReplaceAll a->x:", strings.ReplaceAll(s, "a", "x"))

    // Split dan Join
    parts := strings.Split(s, ", ")
    fmt.Println("Split by ', ':", parts)
    fmt.Println("Join:", strings.Join([]string{"a", "b", "c"}, "-"))

    // Trim
    text := " hello world "
    fmt.Println("TrimSpace:", ""+strings.TrimSpace(text)+"")
    fmt.Println("Trim ' ': ", ""+strings.Trim(text, " ")+"")

    // Count dan Repeat
    fmt.Println("Count 'l':", strings.Count(s, "l"))
    fmt.Println("Repeat 'Go' x3:", strings.Repeat("Go", 3))

    // 3. KONVERSI STRING <-> NUMBER
    fmt.Println("\n=== Konversi ===")
    num := 42
    strNum := strconv.Itoa(num) // Int to string
    fmt.Printf("%d -> '%s'\n", num, strNum)

    parsed, err := strconv.Atoi("100") // String to int
    if err != nil {
        fmt.Println("Error parsing:", err)
    } else {
        fmt.Println("Parsed + 1:", parsed+1)
    }

    // ParseFloat
    floatVal, _ := strconv.ParseFloat("3.14", 64)
    fmt.Println("ParseFloat:", floatVal)

    // 4. STRINGS.BUILDER (performance untuk concatenation)
    fmt.Println("\n=== strings.Builder ===")
    var sb strings.Builder
    sb.WriteString("Ini ")
    sb.WriteString("adalah ")
    sb.WriteString("kalimat ")
    sb.WriteString("panjang")
    fmt.Println("Builder result:", sb.String())
    fmt.Println("Builder len:", sb.Len())
}
```

18. Go Modules (multi-module example)

exercises/18-module/greeter/go.mod

```
// File: 18-module/greeter/go.mod
// Level: Beginner
// Topik: Go Module File
//
// go.mod adalah file konfigurasi module Go.
// Berisi:
// - module name: nama module (bisa di-import oleh module lain)
// - go version: versi Go yang digunakan
// - require: dependensi eksternal
// - replace: mengganti path module (untuk local development)
```

module example.com/greeter

go 1.22

```
// Untuk menambah dependensi:
// go get github.com/gorilla/mux
//
// Untuk update dependensi:
// go mod tidy
```

exercises/18-module/greeter/greeter.go

```
// File: 18-module/greeter/greeter.go
// Level: Beginner
// Topik: Membuat Package
//
// Package greeter adalah contoh pembuatan package sendiri.
// Function yang diawali huruf BESAR bisa diakses dari luar package (exported).
// Function yang diawali huruf kecil hanya bisa diakses dalam package (unexported).
```

package greeter

import "fmt"

```
// Hello adalah exported function (huruf besar)
// Karena exported, bisa dipanggil dari package lain
// Menerima nama dan mengembalikan sapaan
func Hello(name string) string {
    return fmt.Sprintf("Hello, %s!", name)
}
```

```
// Greet adalah exported function
// Menerima slice nama dan menyapa semua
func Greet(names []string) {
    for _, name := range names {
        fmt.Println(Hello(name))
    }
}
```

```
// helloInternal adalah unexported function (huruf kecil)
// Hanya bisa dipanggil dari dalam package greeter
func helloInternal() string {
    return "Internal greeting"
}
```

exercises/18-module/main/go.mod

```
// File: 18-module/main/go.mod
// Level: Beginner
// Topik: Go Module dengan Local Dependensi
//
// go.mod ini memiliki local dependensi ke module greeter.
// "replace" digunakan untuk menunjuk ke path lokal.
// Di production, "replace" diganti dengan version number.
```

module example.com/main

go 1.22

```
// Module main bergantung pada module greeter
require example.com/greeter v0.0.0
```

```
// replace mengarahkan Go ke folder local
// Ini berguna saat development module paralel
replace example.com/greeter => ../greeter
```

```
// Untuk dependensi eksternal (contoh):
// require github.com/gorilla/mux v1.8.1
```

```
// File: 18-module/main/main.go
// Level: Beginner
// Topik: Go Modules
//
// Go Modules adalah sistem manajemen dependensi resmi Go.
// Setiap project Go dimulai dengan `go mod init nama-module`.
//
// Langkah-langkah:
// 1. Buat folder project
// 2. go mod init nama-module
// 3. Tulis kode
// 4. go mod tidy (jika ada dependensi eksternal)
//
// Struktur:
// module-example/
// └── greeter/           # package sendiri
//     ├── go.mod         # module greeter
//     └── greeter.go     # kode package
// └── main/             # program utama
//     ├── go.mod         # module main (depend ke greeter)
//     └── main.go        # entry point

package main

import (
    "fmt"

    // Mengimport package lokal (local module)
    // Path mengikuti module name di go.mod
    "example.com/greeter"
)

func main() {
    // Memanggil function dari package greeter
    // Function yang bisa diakses: diawali huruf BESAR (exported)
    message := greeter.Hello("Anggi")
    fmt.Println(message)

    // Contoh lain
    names := []string{"Anggi", "Budi", "Citra"}
    greeter.Greet(names)
}

// Cara menjalankan:
// cd 18-module/main
// go run main.go
//
// Cara membuat module sendiri:
// cd 18-module/greeter
// go mod init example.com/greeter
//
// Cara menghubungkan main ke greeter:
// cd 18-module/main
// go mod init example.com/main
// go mod edit -replace example.com/greeter=../greeter
// go mod tidy
//
// Untuk dependensi eksternal:
// go get github.com/gorilla/mux
```

Bab 2 — Level Menengah

Tutorial Go Level Menengah

Setelah menguasai dasar-dasar Go, lanjut ke topik menengah untuk membangun aplikasi production-ready.

Topik

#	Topik	Deskripsi	File	Cara Run
1	Context	Timeout, cancellation, passing value antar goroutine	context/main.go	go run context/main.go
2	JSON	Encoding/decoding JSON, struct tags, file I/O	json/main.go	go run json/main.go
3	File I/O	Read/write file, scanner, directory operations	file-io/main.go	go run file-io/main.go
4	Concurrency	WaitGroup, Mutex, Select (multiplexing channel)	concurrency/main.go	go run concurrency/main.go
5	Testing	Table-driven test, benchmark, TestMain, coverage	testing/main.go	go test -v -bench=. -cover
6	Database	SQLite CRUD, transaction, prepared statement	database/main.go	go run database/main.go
7	Logging	Standard log, multi-writer, log ke file	logging/main.go	go run logging/main.go
8	Middleware	HTTP middleware: logging, auth, recovery	middleware/main.go	go run middleware/main.go
9	Env Config	Environment variables, config struct, fallback	env/main.go	go run env/main.go
10	JWT Auth	JWT token create/validate (manual implementasi)	auth/main.go	go run auth/main.go
11	Docker	Multi-stage build, Docker Compose, health check	docker/	docker build -t go-app .

Cara Belajar

- Baca kode di setiap file (ada komentar lengkap)
- Jalankan dengan `go run` atau `go test`
- Modifikasi kode untuk eksperimen
- Pahami konsep, bukan hanya syntax

Prasyarat

Sebelum masuk level menengah, pastikan sudah paham: - ✓ Variabel, fungsi, struct, interface - ✓ Pointer, slice, map - ✓ Goroutine & channel dasar - ✓ Error handling & testing dasar

Instalasi Dependensi

Beberapa topik butuh dependensi eksternal:

```
# Database (SQLite)
cd database && go get modernc.org/sqlite

# WebSocket (expert level)
cd ../websocket && go get github.com/gorilla/websocket

# GraphQL (expert level)
cd ../graphql && go get github.com/graphql-go/graphql
```

| Topik: context


```
// File: middle/context/main.go
// Level: Middle
// Topik: Context
//
// Context digunakan untuk:
// 1. Timeout & Deadline - membatasi waktu eksekusi
// 2. Cancellation - membatalkan operasi yang berjalan
// 3. Passing values - mengirim data antar goroutine (request-scoped)
//
// Context penting untuk HTTP server, database query, dan API calls.
// Selalu gunakan context.Background() atau context.TODO() sebagai parent.

package main

import (
    "context"
    "fmt"
    "time"
)

func main() {
    // 1. CONTEXT WITH TIMEOUT
    // Membuat context yang otomatis selesai setelah 2 detik
    // Berguna untuk: timeout HTTP request, database query
    fmt.Println("=== Context with Timeout ===")
    ctx, cancel := context.WithTimeout(context.Background(), 2*time.Second)
    defer cancel() // selalu panggil cancel untuk membebaskan resource

    // Menjalankan goroutine dengan context
    go task(ctx)

    // Menunggu context selesai (timeout) atau timer 3 detik
    select {
    case <-ctx.Done():
        // ctx.Done() mengembalikan channel yang close saat context selesai
        // ctx.Err() memberikan alasan: DeadlineExceeded atau Canceled
        fmt.Println("Main: context selesai:", ctx.Err())
    case <-time.After(3 * time.Second):
        fmt.Println("Main: timeout tidak terjadi")
    }

    // Beri waktu goroutine mencetak output
    time.Sleep(500 * time.Millisecond)
    fmt.Println()

    // 2. CONTEXT WITH VALUE
    // Untuk passing request-scoped data (seperti userID, requestID)
    // CATATAN: hanya untuk data yang penting untuk request flow
    fmt.Println("=== Context with Value ===")
    ctx2 := context.WithValue(context.Background(), "userID", 12345)
    processRequest(ctx2)
    fmt.Println()

    // 3. CONTEXT WITH CANCEL
    // Manual cancel - menghentikan goroutine dari luar
    fmt.Println("=== Context with Cancel ===")
    ctx3, cancel3 := context.WithCancel(context.Background())
    go cancellableTask(ctx3)

    // Biarkan task berjalan 500ms lalu cancel
    time.Sleep(500 * time.Millisecond)
    cancel3() // mengirim sinyal cancel ke goroutine

    // Beri waktu goroutine merespon cancel
    time.Sleep(200 * time.Millisecond)
}

// task melakukan pekerjaan dengan context timeout
func task(ctx context.Context) {
    select {
    case <-time.After(3 * time.Second):
        // Simulasi pekerjaan yang butuh 3 detik
        fmt.Println("Task selesai (3 detik)")
    case <-ctx.Done():
        // Context selesai (timeout) sebelum task selesai
        fmt.Println("Task dibatalkan karena:", ctx.Err())
    }
}

// processRequest mengambil data dari context
func processRequest(ctx context.Context) {
    // ctx.Value() mengembalikan interface{}, perlu type assertion
    userID := ctx.Value("userID")
    fmt.Println("Processing request untuk user:", userID)
}

// cancellableTask melakukan loop sampai di-cancel
func cancellableTask(ctx context.Context) {
    for {
        select {
        case <-ctx.Done():
            // Menerima sinyal cancel
            fmt.Println("Cancellable task dihentikan")
            return
        default:
            // Melakukan pekerjaan
            fmt.Print(".")
            time.Sleep(100 * time.Millisecond)
        }
    }
}
```

```
middle/json/main.go
```

```
// File: middle/json/main.go
// Level: Middle
// Topik: JSON Encoding/Decoding
//
// Package encoding/json untuk serialisasi data Go ke JSON dan sebaliknya.
// JSON tags pada struct mengontrol nama field di JSON:
// `json:"name"` -> field name
// `json:"name,omitempty"` -> hilangkan jika nilai kosong
// `json:"- "` -> skip field ini

package main

import (
    "encoding/json"
    "fmt"
    "os"
)

// Struct Person dengan JSON tags
// Tag memberitahu encoder/decoder JSON tentang mapping
type Person struct {
    Name string `json:"name"` // field JSON: "name"
    Age int `json:"age"` // field JSON: "age"
    City string `json:"city,omitempty"` // omitempty: hilangkan jika ""
    // Unexported field (huruf kecil) tidak akan di-serialize
    secret string
}

func main() {
    // 1. MARSHAL - Encode Go struct ke JSON string
    fmt.Println("=== Marshal (encode Go -> JSON) ===")
    p := Person{Name: "Anggi", Age: 20, City: "Jakarta"}
    data, err := json.Marshal(p)
    if err != nil {
        panic(err)
    }
    fmt.Println("JSON:", string(data))

    // MarshalIndent untuk pretty print
    dataPretty, _ := json.MarshalIndent(p, "", " ")
    fmt.Println("Pretty JSON:")
    fmt.Println(string(dataPretty))

    // 2. UNMARSHAL - Decode JSON ke Go struct
    fmt.Println("\n=== Unmarshal (decode JSON -> Go) ===")
    jsonStr := `{"name":"Budi","age":25}`
    var p2 Person
    err = json.Unmarshal([]byte(jsonStr), &p2)
    if err != nil {
        panic(err)
    }
    fmt.Printf("Decoded struct: %v\n", p2)

    // 3. MAP[string]interface{} - untuk JSON dinamis
    fmt.Println("\n=== Dynamic JSON with map ===")
    jsonMap := `{"name":"Citra","age":30,"hobbies":["coding","reading"]}`
    var result map[string]interface{}
    json.Unmarshal([]byte(jsonMap), &result)

    // Accessing map values need type assertion
    fmt.Println("Name:", result["name"])
    hobbies := result["hobbies"].([]interface{})
    fmt.Println("Hobbies:")
    for _, h := range hobbies {
        fmt.Println("-", h)
    }

    // 4. JSON ENCODER/DECODER - untuk file I/O
    fmt.Println("\n=== JSON Encoder/Decoder ===")
    file, err := os.Create("data.json")
    if err != nil {
        panic(err)
    }
    defer file.Close()

    // Encoder menulis langsung ke file (lebih efisien untuk file besar)
    encoder := json.NewEncoder(file)
    encoder.Encode(p)
    fmt.Println("Data written to data.json")

    // Decoder membaca dari file
    file2, _ := os.Open("data.json")
    defer file2.Close()

    var p3 Person
    json.NewDecoder(file2).Decode(&p3)
    fmt.Printf("Read from file: %v\n", p3)

    // Cleanup
    os.Remove("data.json")
}
```

Topik: file-io

```
middle/file-io/main.go
```

```
// File: middle/file-io/main.go
// Level: Middle
// Topik: File I/O
//
// Go menyediakan beberapa cara untuk membaca/menulis file:
// 1. os.ReadFile / os.WriteFile - untuk file kecil (all at once)
// 2. bufio.Scanner - membaca baris per baris
```

```
// 3. os.File - untuk operasi file level rendah
// 4. io.Copy - untuk menyalin data antar stream
```

```
package main

import (
    "bufio"    // buffered I/O
    "fmt"
    "os"       // file operations
)

func main() {
    // 1. WRITE FILE - menulis seluruh konten sekaligus
    fmt.Println("=== Write File ===")
    content := "Hello, Go!\nBaris kedua\nBaris ketiga"
    // 0644 = permission: owner r/w, group r, others r
    err := os.WriteFile("output.txt", []byte(content), 0644)
    if err != nil {
        panic(err)
    }
    fmt.Println("File 'output.txt' berhasil ditulis")

    // 2. READ FILE - membaca seluruh konten sekaligus
    fmt.Println("\n=== Read File ===")
    data, err := os.ReadFile("output.txt")
    if err != nil {
        panic(err)
    }
    fmt.Println("File content:")
    fmt.Println(string(data))

    // 3. APPEND TO FILE - menambah konten ke file yang sudah ada
    fmt.Println("\n=== Append to File ===")
    f, err := os.OpenFile("output.txt",
        os.O_APPEND|os.O_WRONLY, 0644)
    if err != nil {
        panic(err)
    }
    defer f.Close()

    f.WriteString("Baris tambahan\n")
    fmt.Println("Baris baru berhasil ditambahkan")

    // 4. READ LINE BY LINE - menggunakan scanner
    fmt.Println("\n=== Read Line by Line ===")
    file, err := os.Open("output.txt")
    if err != nil {
        panic(err)
    }
    defer file.Close()

    scanner := bufio.NewScanner(file)
    lineNum := 1
    for scanner.Scan() {
        // scanner.Text() mengembalikan baris saat ini (tanpa \n)
        fmt.Printf("Baris %d: %s\n", lineNum, scanner.Text())
        lineNum++
    }

    // Cek error scanner
    if err := scanner.Err(); err != nil {
        fmt.Println("Scanner error:", err)
    }

    // 5. FILE INFO - mengecek apakah file ada
    fmt.Println("\n=== File Info ===")
    info, err := os.Stat("output.txt")
    if err == nil {
        fmt.Println("File exists")
        fmt.Println("Size:", info.Size(), "bytes")
        fmt.Println("Permission:", info.Mode())
        fmt.Println("Modified:", info.ModTime())
    } else {
        fmt.Println("File tidak ditemukan")
    }

    // 6. DIRECTORY OPERATIONS
    fmt.Println("\n=== Directory Operations ===")
    os.MkdirAll("test/subdir", 0755)
    fmt.Println("Directory 'test/subdir' dibuat")

    // Rename file
    os.Rename("output.txt", "output_renamed.txt")
    fmt.Println("File di-rename")

    // List files in directory
    entries, _ := os.ReadDir(".")
    fmt.Println("Files in current dir:")
    for _, entry := range entries {
        if entry.IsDir() {
            fmt.Println("[DIR]", entry.Name())
        } else {
            fmt.Println("[FILE]", entry.Name())
        }
    }

    // 7. CLEANUP
    fmt.Println("\n=== Cleanup ===")
    os.Remove("output_renamed.txt")
    os.RemoveAll("test")
    fmt.Println("Cleanup selesai")
}
```

```
// File: middle/concurrency/main.go
// Level: Middle
// Topik: Concurrency Lanjutan - WaitGroup, Mutex, Select
//
// Lanjutan dari goroutine & channel di level Beginner.
// Pola-pola penting:
// 1. WaitGroup - menunggu goroutine selesai
// 2. Mutex - mengamankan akses ke shared data
// 3. Select - multiplexing multiple channels

package main

import (
    "fmt"
    "sync" // WaitGroup, Mutex
    "time"
)

func main() {
    // 1. WAITGROUP - menunggu kumpulan goroutine selesai
    // WaitGroup seperti counter: Add() menambah, Done() mengurangi, Wait() menunggu
    fmt.Println("=== WaitGroup ===")
    var wg sync.WaitGroup

    // Menjalankan 5 worker goroutine
    for i := 0; i < 5; i++ {
        wg.Add(1) // increment counter
        go worker(i, &wg)
    }

    wg.Wait() // blocking sampai counter = 0
    fmt.Println("Semua worker selesai")
    fmt.Println()

    // 2. MUTEX - mutual exclusion
    // Mencegah race condition ketika multiple goroutine mengakses data yang sama
    fmt.Println("=== Mutex ===")
    var counter int
    var mu sync.Mutex

    // 10 goroutine mencoba increment counter secara concurrent
    for i := 0; i < 10; i++ {
        wg.Add(1)
        go increment(&counter, &mu, &wg)
    }

    wg.Wait()
    fmt.Println("Counter akhir:", counter) // harus 10
    fmt.Println()

    // 3. SELECT - multiplexing channel
    // Menunggu multiple channel, merespon yang pertama siap
    fmt.Println("=== Select ===")
    c1 := make(chan string)
    c2 := make(chan string)

    // Goroutine 1: kirim setelah 100ms
    go func() {
        time.Sleep(100 * time.Millisecond)
        c1 <- "dari channel 1"
    }()

    // Goroutine 2: kirim setelah 200ms
    go func() {
        time.Sleep(200 * time.Millisecond)
        c2 <- "dari channel 2"
    }()

    // Select: merespon channel yang pertama siap
    for i := 0; i < 2; i++ {
        select {
            case msg1 := <- c1:
                fmt.Println(msg1)
            case msg2 := <- c2:
                fmt.Println(msg2)
            case <-time.After(500 * time.Millisecond):
                // Timeout jika tidak ada channel yang siap dalam 500ms
                fmt.Println("Timeout!")
        }
    }

}

// worker mensimulasikan pekerjaan goroutine
func worker(id int, wg *sync.WaitGroup) {
    // Done() harus dipanggil, gunakan defer untuk keamanan
    defer wg.Done()

    time.Sleep(100 * time.Millisecond) // simulasi kerja
    fmt.Printf("Worker %d selesai\n", id)
}

// increment menambah counter dengan aman menggunakan Mutex
func increment(counter *int, mu *sync.Mutex, wg *sync.WaitGroup) {
    defer wg.Done()

    // Lock: goroutine lain harus menunggu sampai Unlock
    mu.Lock()
    *counter++ // critical section - hanya satu goroutine boleh akses
    mu.Unlock()
    // Unlock: goroutine lain bisa melanjutkan
}
```

```
// File: middle/testing/main.go
// Level: Middle
// Topik: Testing Lanjutan
//
// Go memiliki package "testing" built-in yang powerful.
// Fitur:
// 1. Table-driven tests - test dengan multiple test cases
// 2. Benchmark - mengukur performa kode
// 3. TestMain - setup/teardown untuk semua test
// 4. Coverage - persentase kode yang di-test
//
// NOTE: File ini berisi fungsi yang akan di-test DAN fungsi test.
// Untuk menjalankan, rename file jadi *_test.go atau gunakan go test -run.

package main

import "testing"

// ===== FUNGSI YANG AKAN DI-TEST =====

// Tambah menjumlahkan dua bilangan
func Tambah(a, b int) int {
    return a + b
}

// IsEven mengecek apakah bilangan genap
func IsEven(n int) bool {
    return n%2 == 0
}

// ===== UNIT TEST =====

// TestTambah adalah contoh table-driven test
// Table-driven test: mendefinisikan test cases dalam slice struct
func TestTambah(t *testing.T) {
    // Test cases dalam bentuk slice struct
    tests := []struct {
        name    string // nama test case
        a, b    int   // input
        expected int   // expected output
    }{
        {name: "positive numbers", a: 2, b: 3, expected: 5},
        {name: "negative numbers", a: -1, b: 1, expected: 0},
        {name: "zero", a: 0, b: 0, expected: 0},
        {name: "large numbers", a: 1000, b: 2000, expected: 3000},
    }

    // Iterasi test cases
    for _, tt := range tests {
        // t.Run() menjalankan sub-test dengan nama
        t.Run(tt.name, func(t *testing.T) {
            result := Tambah(tt.a, tt.b)
            if result != tt.expected {
                // t.Errorf melaporkan error tapi melanjutkan test
                t.Errorf("Tambah(%d, %d) = %d; want %d",
                    tt.a, tt.b, result, tt.expected)
            }
        })
    }
}

// TestIsEven adalah test sederhana
func TestIsEven(t *testing.T) {
    if !IsEven(2) {
        t.Error("2 harus even")
    }
    if IsEven(3) {
        t.Error("3 harus odd")
    }
}

// ===== BENCHMARK =====

// BenchmarkTambah mengukur performa fungsi Tambah
// Benchmark dijalankan dengan: go test -bench=.
// b.N diatur oleh Go untuk mendapatkan hasil yang stabil
func BenchmarkTambah(b *testing.B) {
    // Loop b.N kali - jumlah loop diatur oleh Go
    for i := 0; i < b.N; i++ {
        Tambah(100, 200)
    }
}

// ===== TEST MAIN =====

// TestMain adalah entry point untuk semua test dalam package
// Berguna untuk setup/teardown global
func TestMain(m *testing.M) {
    // Setup: dijalankan sebelum semua test
    println("=== SETUP: Membuat database test ===")

    // m.Run() menjalankan semua test
    code := m.Run()

    // Teardown: dijalankan setelah semua test selesai
    println("=== TEARDOWN: Membersihkan database test ===")

    // os.Exit() dengan kode dari test
    // NOTE: untuk actual code, gunakan os.Exit(code)
    println("Exit code:", code)
}

// ===== RUNNING TESTS =====
// go test -v          -> verbose, lihat detail setiap test
// go test -run TestTambah -> jalankan test spesifik
// go test -bench=.    -> jalankan benchmark
```

```
// go test -bench=Bench      -> jalankan semua benchmark
// go test -cover            -> lihat code coverage
// go test -coverprofile=coverage.out -> output coverage ke file
```

Topik: database

middle/database/main.go

```
// File: middle/database/main.go
// Level: Middle
// Topik: Database SQL dengan SQLite
//
// Go memiliki package "database/sql" - interface standar untuk database SQL.
// Dibutuhkan driver spesifik untuk tiap database:
// - SQLite: modernc.org/sqlite (pure Go, no C compiler needed)
// - PostgreSQL: github.com/lib/pq
// - MySQL: github.com/go-sql-driver/mysql
//
// Step install:
// cd middle/database
// go get modernc.org/sqlite

package main

import (
    "database/sql"
    "fmt"
    "log"

    // Import driver SQLite (side effect: menggunakan _)
    _ "modernc.org/sqlite"
)

// User merepresentasikan tabel users
type User struct {
    ID      int
    Name    string
    Email   string
}

func main() {
    // 1. KONEKSI DATABASE
    // sql.Open() tidak langsung connect, hanya menyiapkan koneksi
    db, err := sql.Open("sqlite", "test.db")
    if err != nil {
        log.Fatal("Gagal buka database:", err)
    }
    defer db.Close() // tutup koneksi saat fungsi selesai

    // Test koneksi
    err = db.Ping()
    if err != nil {
        log.Fatal("Gagal koneksi:", err)
    }
    fmt.Println("Koneksi database berhasil")

    // Cleanup tabel jika ada
    defer db.Exec("DROP TABLE IF EXISTS users")

    // 2. CREATE TABLE
    _, err = db.Exec(`CREATE TABLE IF NOT EXISTS users (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT NOT NULL,
        email TEXT UNIQUE NOT NULL
    )`)
    if err != nil {
        log.Fatal("Gagal create table:", err)
    }
    fmt.Println("Tabel users siap")
    fmt.Println()

    // 3. INSERT
    fmt.Println("=== INSERT ===")
    result, err := db.Exec(
        "INSERT INTO users (name, email) VALUES (?, ?)",
        "Anggi", "anggi@mail.com",
    )
    if err != nil {
        log.Fatal("Gagal insert:", err)
    }
    id, _ := result.LastInsertId() // ambil ID yang baru di-generate
    fmt.Println("Inserted user ID:", id)

    // Insert multiple dalam 1 transaction
    fmt.Println("Insert multiple dengan transaction:")
    tx, _ := db.Begin() // mulai transaction
    tx.Exec("INSERT INTO users (name, email) VALUES (?, ?)", "Budi", "budi@mail.com")
    tx.Exec("INSERT INTO users (name, email) VALUES (?, ?)", "Citra", "citra@mail.com")
    tx.Commit() // commit transaction
    fmt.Println("Transaction selesai")
    fmt.Println()

    // 4. QUERY SINGLE ROW
    fmt.Println("=== Query Single Row ===")
    var u User
    // QueryRow mengambil baris pertama
    row := db.QueryRow("SELECT id, name, email FROM users WHERE id = ?", 1)
    // Scan membaca hasil query ke variabel (urutan harus sesuai SELECT)
    err = row.Scan(&u.ID, &u.Name, &u.Email)
    if err != nil {
        log.Fatal("Gagal query:", err)
    }
    fmt.Printf("User: id=%d, name=%s, email=%s\n", u.ID, u.Name, u.Email)
    fmt.Println()
}
```

```

// 5. QUERY MULTIPLE ROWS
fmt.Println("=== Query Multiple Rows ===")
rows, err := db.Query("SELECT id, name, email FROM users ORDER BY id")
if err != nil {
    log.Fatal("Gagal query:", err)
}
defer rows.Close() // tutup rows saat selesai

// Iterasi hasil query
for rows.Next() {
    var user User
    err := rows.Scan(&user.ID, &user.Name, &user.Email)
    if err != nil {
        log.Fatal("Gagal scan:", err)
    }
    fmt.Printf(" id=%d, name=%s, email=%s\n", user.ID, user.Name, user.Email)
}
// Cek error setelah iterasi
if err = rows.Err(); err != nil {
    log.Fatal("Error rows:", err)
}
fmt.Println()

// 6. UPDATE
fmt.Println("=== UPDATE ===")
db.Exec("UPDATE users SET name = ? WHERE id = ?", "Anggi Updated", 1)
fmt.Println("User 1 updated")

// 7. DELETE
fmt.Println("\n=== DELETE ===")
db.Exec("DELETE FROM users WHERE id = ?", 3)
fmt.Println("User 3 deleted")

// 8. COUNT
fmt.Println("\n=== COUNT ===")
var count int
db.QueryRow("SELECT COUNT(*) FROM users").Scan(&count)
fmt.Println("Total users tersisa:", count)
}

```

Topik: logging

```
// File: middle/logging/main.go
// Level: Middle
// Topik: Logging
//
// Package "log" adalah logging bawaan Go.
// Level: Print (info), Fatal (exit), Panic (panic)
// Format: log.SetFlags() untuk konfigurasi format
// Logger bisa di-custom: prefix, output destination, format
//
// Untuk production, gunakan library seperti logrus atau zap:
// go get github.com/sirupsen/logrus
// go get go.uber.org/zap

package main

import (
    "io"           // io.MultiWriter
    "log"          // standard logger
    "os"           // file operations
    "time"         // time formatting
)

func main() {
    // 1. BASIC LOG
    fmt := log.New(os.Stdout, "", 0) // hanya untuk fmt.Println-style
    fmt.Println("=== Basic Log ===")
    log.Println("Ini log biasa")
    log.Printf("User %s login pada %s\n", "Anggi", time.Now().Format(time.RFC3339))

    // 2. LOG DENGAN FORMAT
    fmt.Println("\n=== Log Format ===")
    // log.Ldate: tanggal, Ltime: waktu, Lshortfile: file & line number
    warnLog := log.New(os.Stdout,
        "WARN: ",
        log.Ldate|log.Ltime|log.Lshortfile,
    )
    warnLog.Println("Ini warning dengan lokasi file")
    warnLog.Println("Format: tanggal + waktu + file:line")

    // 3. LOG KE FILE
    fmt.Println("\n=== Log to File ===")
    file, err := os.Create("app.log")
    if err != nil {
        log.Fatal(err)
    }
    defer file.Close()
    defer os.Remove("app.log")

    fileLog := log.New(file, "INFO: ", log.LstdFlags)
    fileLog.Println("Aplikasi dimulai")
    fileLog.Println("Aplikasi berjalan")
    fileLog.Println("Aplikasi selesai")

    // 4. MULTI WRITER - log ke stdout dan file sekaligus
    fmt.Println("\n=== Multi Writer ===")
    multiWriter := io.MultiWriter(os.Stdout, file)
    multiLog := log.New(multiWriter, "MULTI: ", log.LstdFlags)
    multiLog.Println("Log ini muncul di stdout DAN file")
    multiLog.Println("Berguna untuk development sekaligus logging ke file")

    // 5. FATAL & PANIC (HATI-HATI!)
    // log.Fatal: mencetak log lalu exit(1)
    // log.Panic: mencetak log lalu panic
    // Uncomment untuk melihat efek:
    // log.Fatal("Fatal error - program akan exit!")
    // log.Panic("Panic error - program akan panic!")
}
```

Topik: middleware


```
// File: middle/middleware/main.go
// Level: Middle
// Topik: Middleware Pattern pada HTTP Server
//
// Middleware adalah fungsi yang membungkus HTTP handler.
// Berguna untuk: logging, auth, recovery, rate limiting, CORS, dll.
// Pola: middleware(http.Handler) http.Handler
//
// Middleware bisa di-chain: handler = m1(m2(m3(handler)))
// Atau dibalik: handler = chain(handler, m1, m2, m3)

package main

import (
    "fmt"
    "log"
    "net/http"
    "time"
)

// loggingMiddleware mencatat setiap request (method, path, durasi)
func loggingMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        // Before handler
        start := time.Now()

        // Panggil handler asli
        next.ServeHTTP(w, r)

        // After handler
        log.Printf("[%s] %s %s %v",
            r.Method, r.RequestURI, r.RemoteAddr, time.Since(start))
    })
}

// authMiddleware memvalidasi token dari header Authorization
func authMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        // Ambil token dari header
        token := r.Header.Get("Authorization")

        // Validasi (sederhana)
        if token != "valid-token" {
            http.Error(w, "Unauthorized", http.StatusUnauthorized)
            return // stop: jangan panggil handler berikutnya
        }

        // Token valid, lanjut ke handler
        next.ServeHTTP(w, r)
    })
}

// recoveryMiddleware menangkap panic agar server tidak crash
func recoveryMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        defer func() {
            if err := recover(); err != nil {
                // Log error tanpa crash
                log.Printf("Recovered from panic: %v", err)
                http.Error(w, "Internal Server Error",
                    http.StatusInternalServerError)
            }
        }()
        next.ServeHTTP(w, r)
    })
}

// chainMiddlewares menerapkan middleware secara berurutan
// Urutan penting: recovery harus paling luar
func chainMiddlewares(h http.Handler, middlewares ...func(http.Handler) http.Handler) http.Handler {
    for _, m := range middlewares {
        h = m(h)
    }
    return h
}

// Handler: mengembalikan JSON response
func helloHandler(w http.ResponseWriter, r *http.Request) {
    fmt.Fprintln(w, `{"message": "Hello, World!"}`)
}

// Handler: sengaja panic untuk demo recovery
func panicHandler(w http.ResponseWriter, r *http.Request) {
    panic("Terjadi error tak terduga!")
}

func main() {
    mux := http.NewServeMux()
    mux.HandleFunc("/hello", helloHandler)
    mux.HandleFunc("/panic", panicHandler)

    // Chain middlewares
    handler := chainMiddlewares(mux,
        recoveryMiddleware, // paling luar
        loggingMiddleware,
        authMiddleware, // paling dalam (dijalankan pertama sebelum handler)
    )

    log.Println("Server running on :8080")
    log.Println("Test: curl -H 'Authorization: valid-token' localhost:8080/hello")
    log.Println("Test: curl -H 'Authorization: valid-token' localhost:8080/panic")
    http.ListenAndServe(":8080", handler)
}
```

Topik: env

📄 middle/env/main.go

```
// File: middle/env/main.go
// Level: Middle
// Topik: Environment Variables & Konfigurasi
//
// Environment variables (env vars) adalah cara standard untuk:
// 1. Konfigurasi environment (development/production)
// 2. Menyimpan secrets (API keys, database password)
// 3. Konfigurasi port, database path, dll
//
// Go: os.Getenv() untuk membaca, os.Setenv() untuk menulis
// Untuk production: go get github.com/joho/godotenv

package main

import (
    "fmt"
    "os"
    "strconv"
)

// Config menyimpan semua konfigurasi aplikasi
type Config struct {
    Port      int    // Port HTTP server
    DBPath    string // Path file database
    Env       string // Environment (dev/staging/prod)
    Debug     bool   // Mode debug
}

// getEnv membaca env var dengan fallback value
// Jika variabel tidak ada, gunakan fallback
func getEnv(key, fallback string) string {
    if val := os.Getenv(key); val != "" {
        return val
    }
    return fallback
}

// loadConfig membaca semua env var dan mengembalikan Config
func loadConfig() Config {
    port, _ := strconv.Atoi(getEnv("PORT", "8080"))
    debug, _ := strconv.ParseBool(getEnv("DEBUG", "false"))

    return Config{
        Port:    port,
        DBPath:  getEnv("DB_PATH", "./data.db"),
        Env:     getEnv("ENV", "development"),
        Debug:   debug,
    }
}

func main() {
    // 1. SET ENV VARS (simulasi)
    // Di production, env var di-set di Docker/docker-compose/CI
    fmt.Println("=== Set Environment Variables ===")
    os.Setenv("PORT", "3000")
    os.Setenv("ENV", "production")
    os.Setenv("DEBUG", "true")
    os.Setenv("DB_PATH", "/app/data/production.db")

    // 2. BACA CONFIG
    fmt.Println("\n=== Load Config ===")
    config := loadConfig()
    fmt.Printf("Config: %v\n", config)
    fmt.Printf("Server akan mulai di port %d (%s mode)\n",
        config.Port, config.Env)

    if config.Debug {
        fmt.Println("Debug mode: ON - log akan lebih detail")
    }

    // 3. BACA SINGLE ENV
    fmt.Println("\n=== Read Single Env ===")
    home := os.Getenv("HOME")
    user := os.Getenv("USER")
    fmt.Printf("Home: %s\n", home)
    fmt.Printf("User: %s\n", user)

    // 4. LIST ALL ENV VARS
    fmt.Println("\n=== All Env Vars (filtered) ===")
    for _, env := range os.Environ() {
        // Tampilkan hanya env vars yang relevan
        if len(env) > 20 {
            fmt.Println(env[:20], "...")
        }
    }

    // 5. UNSET - membersihkan (untuk production: jangan set lewat kode)
    os.Unsetenv("PORT")
    os.Unsetenv("ENV")
    os.Unsetenv("DEBUG")
    os.Unsetenv("DB_PATH")
    fmt.Println("\nEnv vars cleaned up")
}
```

Topik: auth

📄 middle/auth/main.go

```
// File: middle/auth/main.go
// Level: Middle
```

```
// Topik: JWT Authentication (Implementasi Manual)
//
// JWT (JSON Web Token) format: header.payload.signature
// 1. Header: algoritma & tipe token
// 2. Payload: data (userID, username, exp)
// 3. Signature: verifikasi token tidak dimodifikasi
//
// Implementasi ini untuk PEMBELAJARAN.
// Untuk production: go get github.com/golang-jwt/jwt/v5

package main

import (
    "crypto/hmac"
    "crypto/sha256"
    "encoding/base64"
    "encoding/json"
    "fmt"
    "net/http"
    "strings"
    "time"
)

// jwtSecret adalah kunci rahasia untuk sign & verify token
// Dalam production: simpan di environment variable
var jwtSecret = []byte("rahasia-kunci-jwt-2024")

// createToken membuat JWT token baru
// Format: base64URL(header).base64URL(payload).base64URL(signature)
func createToken(userID int, username string) string {
    // Header: algoritma HS256 & tipe JWT
    header := `{"alg":"HS256","typ":"JWT"}`
    // Payload: data user + expired time (24 jam dari sekarang)
    payload := fmt.Sprintf(
        `{"user_id":%d,"username":"%s","exp":%d}`,
        userID, username, time.Now().Add(24*time.Hour).Unix(),
    )

    // Encode header & payload ke base64 URL-safe
    encodedHeader := base64.URLEncode([]byte(header))
    encodedPayload := base64.URLEncode([]byte(payload))

    // Buat signing input: header.payload
    signingInput := encodedHeader + "." + encodedPayload
    // Sign menggunakan HMAC-SHA256
    signature := sign(signingInput)

    // Token final: header.payload.signature
    return signingInput + "." + signature
}

// sign membuat signature menggunakan HMAC-SHA256
func sign(input string) string {
    mac := hmac.New(sha256.New, jwtSecret)
    mac.Write([]byte(input))
    return base64.URLEncode(mac.Sum(nil))
}

// base64URLEncode encode ke base64 tanpa padding (=)
func base64URLEncode(data []byte) string {
    return strings.TrimRight(base64.URLEncoding.EncodeToString(data), "=")
}

// validateToken memverifikasi token dan mengembalikan payload
func validateToken(token string) (map[string]interface{}, error) {
    // Split token menjadi 3 bagian
    parts := strings.Split(token, ".")
    if len(parts) != 3 {
        return nil, fmt.Errorf("format token tidak valid")
    }

    // Verifikasi signature
    signingInput := parts[0] + "." + parts[1]
    expectedSig := sign(signingInput)
    if parts[2] != expectedSig {
        return nil, fmt.Errorf("signature tidak valid")
    }

    // Decode payload
    payloadBytes, err := base64.URLEncoding.WithPadding(base64.NoPadding).DecodeString(parts[1])
    if err != nil {
        return nil, fmt.Errorf("gagal decode payload")
    }

    // Parse payload ke map
    var payload map[string]interface{}
    json.Unmarshal(payloadBytes, &payload)

    // Verifikasi expired
    exp := payload["exp"].(float64)
    if time.Now().Unix() > int64(exp) {
        return nil, fmt.Errorf("token sudah expired")
    }

    return payload, nil
}

// authHandler - endpoint yang membutuhkan auth
func authHandler(w http.ResponseWriter, r *http.Request) {
    // Ambil token dari header Authorization: Bearer <token>
    token := r.Header.Get("Authorization")
    token = strings.TrimPrefix(token, "Bearer ")

    payload, err := validateToken(token)
    if err != nil {
        http.Error(w, "Unauthorized: "+err.Error(), http.StatusUnauthorized)
        return
    }
}
```

```

        fmt.Fprintf(w, "Welcome %s! (User ID: %.0f)\n",
            payload["username"], payload["user_id"])
    }

    // loginHandler - endpoint untuk mendapatkan token
    func loginHandler(w http.ResponseWriter, r *http.Request) {
        // Di production: validasi username & password dari database
        token := createToken(1, "Anggi")
        w.Header().Set("Content-Type", "application/json")
        fmt.Fprintf(w, `{"token": "%s"}`, token)
    }

    func main() {
        http.HandleFunc("/login", loginHandler)
        http.HandleFunc("/protected", authHandler)

        fmt.Println("Auth server running on :8080")
        fmt.Println("\nCoba:")
        fmt.Println("1. curl localhost:8080/login")
        fmt.Println("2. TOKEN=$(curl -s localhost:8080/login | grep -o '\"token\": \"[^\"]*\"' | cut -d'\"' -f4)")
        fmt.Println("3. curl -H 'Authorization: Bearer $TOKEN' localhost:8080/protected")
        http.ListenAndServe(":8080", nil)
    }
}

```

Topik: docker

middle/docker/Dockerfile

```

FROM golang:1.22-alpine AS builder
WORKDIR /app
COPY . .
RUN go build -o main .

FROM alpine:latest
RUN apk --no-cache add ca-certificates
WORKDIR /root/
COPY --from=builder /app/main .
EXPOSE 8080
CMD ["/main"]

```

middle/docker/docker-compose.yml

```

version: '3.8'

services:
  app:
    build: .
    ports:
      - "8080:8080"
    environment:
      - PORT=8080
      - ENV=production
    volumes:
      - ./data:/root/data
    restart: unless-stopped

    # Example with postgres
  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: goapp
      POSTGRES_USER: goapp
      POSTGRES_PASSWORD: secret
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data

volumes:
  pgdata:

```

```
// File: middle/docker/main.go
// Level: Middle
// Topik: Aplikasi Go untuk Docker
//
// Aplikasi ini di-build menjadi Docker image.
// Dockerfile menggunakan multi-stage build:
// Stage 1: build binary (golang:alpine)
// Stage 2: run binary (alpine minimal)
//
// Keuntungan multi-stage: image size kecil (~15MB vs ~800MB)

package main

import (
    "fmt"
    "log"
    "net/http"
    "os"
)

func main() {
    // Baca port dari environment variable
    // Convention: PORT env var (standard di Heroku, Railway, dll)
    port := os.Getenv("PORT")
    if port == "" {
        port = "8080" // default port
    }

    // Health check endpoint - penting untuk Docker/Kubernetes
    http.HandleFunc("/health", func(w http.ResponseWriter, r *http.Request) {
        w.Header().Set("Content-Type", "application/json")
        fmt.Fprint(w, `{"status":"healthy","service":"go-app"}`)
    })

    // Root endpoint
    http.HandleFunc("/", func(w http.ResponseWriter, r *http.Request) {
        hostname, _ := os.Hostname()
        fmt.Fprintf(w,
            "Hello from Dockerized Go App!\n\n"+
            "Hostname: %s\n"+
            "Port: %s\n"+
            "API Base: http://localhost:%s\n",
            hostname, port, port,
        )
    })

    log.Printf("Server running on :%s", port)
    log.Fatal(http.ListenAndServe(":"+port, nil))
}

/*
Cara build & run:

1. Build Docker image:
   cd fullstack/middle/docker
   docker build -t go-app .

2. Run container:
   docker run -p 8080:8080 go-app

3. Test:
   curl localhost:8080
   curl localhost:8080/health

4. Docker Compose (multi-service):
   docker compose up -d
*/
```

Tutorial Go Level Mahir

Level mahir mencakup topik-topik production-grade: arsitektur microservices, deployment, CI/CD, dan optimisasi.

Topik

#	Topik	Deskripsi	File	Cara Run
1	gRPC	High-performance RPC dengan Protocol Buffers	grpc/proto/todo.proto	protoc --go_out=. --go-grpc_out=. proto/todo.proto
2	Kubernetes	Deployment, Service, HPA (auto-scaling)	kubernetes/deployment.yaml	kubectl apply -f deployment.yaml
3	CI/CD	GitHub Actions: test → build → docker → deploy	cicd/github/workflows/go.yml	Push ke GitHub → otomatis jalan
4	Profiling	Pprof, race detector, benchmark, fan-out pattern	profiling/main.go	go run profiling/main.go
5	Design Patterns	Repository, Factory, Builder, Singleton di Go	design-patterns/main.go	go run design-patterns/main.go
6	WebSocket	Real-time bidirectional communication	websocket/main.go	go run websocket/main.go
7	GraphQL	Flexible API query language	graphql/main.go	go run graphql/main.go

Instalasi Dependensi

```
# gRPC tools
go install google.golang.org/protobuf/cmd/protoc-gen-go@latest
go install google.golang.org/grpc/cmd/protoc-gen-go-grpc@latest

# WebSocket
cd websocket && go get github.com/gorilla/websocket

# GraphQL
cd graphql && go get github.com/graphql-go/graphql
```

Commands Penting

```
# Profiling
go test -bench=. -cpuprofile=cpu.out -memprofile=mem.out
go tool pprof -http=:8081 cpu.out

# Race detection
go run -race main.go
go test -race ./...

# Build untuk production
GOOS=linux GOARCH=amd64 go build -ldflags="-s -w" -o app .
# ldflags -s -w: strip debug info (smaller binary)

# WebAssembly (eksekusi Go di browser)
GOOS=js GOARCH=wasm go build -o main.wasm
```

Prasyarat

Sebelum masuk level mahir, pastikan sudah paham: - ✓ Semua topik beginner (exercises/) - ✓ Semua topik middle (context, testing, middleware, database) - ✓ Docker dasar - ✓ HTTP & REST API

Topik: grpc

```
syntax = "proto3";

package todo;

option go_package = "todo/";

service TodoService {
  rpc AddTodo (AddTodoRequest) returns (Todo);
  rpc ListTodos (Empty) returns (TodoList);
  rpc GetTodo (GetTodoRequest) returns (Todo);
}

message Todo {
  int32 id = 1;
  string title = 2;
  bool done = 3;
}

message AddTodoRequest {
  string title = 1;
}

message GetTodoRequest {
  int32 id = 1;
}

message TodoList {
  repeated Todo todos = 1;
}

message Empty {}

// Generate:
// go install google.golang.org/protobuf/cmd/protoc-gen-go@latest
// go install google.golang.org/grpc/cmd/protoc-gen-go-grpc@latest
// protoc --go_out=. --go-grpc_out=. proto/todo.proto
```

Topik: kubernetes

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: go-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: go-app
  template:
    metadata:
      labels:
        app: go-app
    spec:
      containers:
        - name: go-app
          image: go-app:latest
          ports:
            - containerPort: 8080
          env:
            - name: PORT
              value: "8080"
            - name: ENV
              value: "production"
          resources:
            requests:
              memory: "64Mi"
              cpu: "100m"
            limits:
              memory: "128Mi"
              cpu: "200m"
          livenessProbe:
            httpGet:
              path: /health
              port: 8080
          readinessProbe:
            httpGet:
              path: /ready
              port: 8080
---
apiVersion: v1
kind: Service
metadata:
  name: go-app-service
spec:
  selector:
    app: go-app
  ports:
    - port: 80
      targetPort: 8080
  type: LoadBalancer
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: go-app-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: go-app
  minReplicas: 3
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
```

Topik: cicd


```
name: CI/CD Pipeline

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Setup Go
        uses: actions/setup-go@v5
        with:
          go-version: '1.22'
      - name: Install dependencies
        run: go mod download
      - name: Run tests
        run: go test ./... -v -cover
      - name: Run linter
        run: |
          go install golang.org/x/lint/golint@latest
          golint ./...

  build:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Build binary
        run: GOOS=linux GOARCH=amd64 go build -o app .
      - name: Upload artifact
        uses: actions/upload-artifact@v4
        with:
          name: app
          path: ./app

  docker:
    needs: build
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Login to GitHub Container Registry
        uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${ github.actor }
          password: ${ secrets.GITHUB_TOKEN }
      - name: Build and push Docker image
        uses: docker/build-push-action@v5
        with:
          push: true
          tags: ghcr.io/${ github.repository }:latest
```

Topik: profiling

```
// File: expert/profiling/main.go
// Level: Expert
// Topik: Profiling & Optimization Patterns
//
// Profiling adalah proses mengukur performa aplikasi:
// - CPU usage: fungsi mana yang paling boros CPU
// - Memory usage: berapa banyak memory yang dipakai
// - Goroutine: jumlah goroutine yang berjalan
//
// Go profiling tools:
// 1. pprof: CPU & Memory profiler
// 2. trace: Goroutine tracing
// 3. race detector: Data race detection
//
// Commands:
// go test -bench=. -cpuprofile=cpu.out -memprofile=mem.out
// go tool pprof cpu.out
// go run -race main.go
```

```
package main
```

```
import (
    "fmt"
    "sync"
    "time"
)
```

```
// ===== CONCURRENCY PATTERNS =====
```

```
// 1. SEQUENTIAL VS CONCURRENT
// Demonstrasi perbedaan waktu eksekusi
```

```
// Post struct
type Post struct {
    ID    int
    Body  string
}
```

```
// Comment struct
type Comment struct {
    ID      int
    Content string
}
```

```

// fetchPost simulasi API call (100ms)
func fetchPost(id int) Post {
    time.Sleep(100 * time.Millisecond)
    return Post{ID: id, Body: "Post content"}
}

// fetchComments simulasi API call (100ms)
func fetchComments(postID int) []Comment {
    time.Sleep(100 * time.Millisecond)
    return []Comment{
        {ID: 1, Content: "Comment 1"},
        {ID: 2, Content: "Comment 2"},
    }
}

// Sequential: post diambil, baru comment (total: ~200ms)
func renderPageSequential(postID int) time.Duration {
    start := time.Now()
    post := fetchPost(postID)
    comments := fetchComments(post.ID)
    elapsed := time.Since(start)
    fmt.Printf("Sequential - %d comments, waktu: %v\n",
        len(comments), elapsed)
    return elapsed
}

// Concurrent: post dan comment diambil bersamaan (total: ~100ms)
func renderPageConcurrent(postID int) time.Duration {
    start := time.Now()

    // Struct untuk menampung hasil goroutine
    type result struct {
        post      Post
        comments []Comment
    }

    // Channel untuk hasil
    ch := make(chan result)

    // Jalankan goroutine
    go func() {
        ch <- result{
            fetchPost(postID),
            fetchComments(postID),
        }
    }()

    res := <-ch
    elapsed := time.Since(start)
    fmt.Printf("Concurrent - %d comments, waktu: %v\n",
        len(res.comments), elapsed)
    return elapsed
}

// 2. FAN-OUT PATTERN
// Membagi pekerjaan ke beberapa worker goroutine
func fanOutExample() {
    fmt.Println("\n=== Fan-Out Pattern ===")
    items := []int{1, 2, 3, 4, 5, 6, 7, 8, 9, 10}

    start := time.Now()
    results := processFanOut(items, 3) // 3 workers
    fmt.Printf("Hasil: %v\n", results)
    fmt.Printf("Waktu: %v (sequential akan ~%dms)\n",
        time.Since(start), len(items)*10)
}

// processFanOut memproses items dengan multiple workers
func processFanOut(items []int, numWorkers int) []int {
    // Buat channel untuk jobs dan results
    jobs := make(chan int, len(items))
    results := make(chan int, len(items))

    // Start workers
    var wg sync.WaitGroup
    for i := 0; i < numWorkers; i++ {
        wg.Add(1)
        go workerFn(i, jobs, results, &wg)
    }

    // Kirim jobs ke channel
    for _, item := range items {
        jobs <- item
    }
    close(jobs) // tidak ada job lagi

    // Tunggu semua worker selesai
    wg.Wait()
    close(results) // tidak ada result lagi

    // Kumpulkan hasil
    var output []int
    for result := range results {
        output = append(output, result)
    }
    return output
}

// workerFn adalah worker yang memproses job
func workerFn(id int, jobs <-chan int, results chan<- int, wg *sync.WaitGroup) {
    defer wg.Done()
    for job := range jobs {
        // Simulasi kerja
        time.Sleep(10 * time.Millisecond)
        fmt.Printf("Worker %d memproses: %d\n", id, job)
        results <- job * 2
    }
}

```

```
// ===== MAIN =====

func main() {
    // 1. Sequential vs Concurrent
    fmt.Println("=== Sequential vs Concurrent ===")
    seqTime := renderPageSequential(1)
    concTime := renderPageConcurrent(1)
    fmt.Printf("Percepatan: %.fx\n", float64(seqTime)/float64(concTime))

    // 2. Fan-Out Pattern
    fanOutExample()

    fmt.Println("\n=== Tips Optimization ===")
    fmt.Println("1. Gunakan goroutine untuk I/O bound tasks")
    fmt.Println("2. Gunakan channel untuk komunikasi")
    fmt.Println("3. Hindari data race: gunakan mutex atau channel")
    fmt.Println("4. Profiling: go test -bench=.")
    fmt.Println("5. Race detector: go run -race main.go")
    fmt.Println("6. Pprof: go tool pprof cpu.out")
}
```

Topik: design-patterns

📁 expert/design-patterns/main.go

```
// File: expert/design-patterns/main.go
// Level: Expert
// Topik: Design Patterns di Go
//
// Design patterns adalah solusi umum untuk masalah umum dalam programming.
// Go tidak mewarisi pola OOP klasik (inheritance), tapi punya cara sendiri.
//
// Pola-pola yang umum di Go:
// 1. Repository Pattern - abstraksi database
// 2. Factory Pattern - membuat objek berdasarkan kondisi
// 3. Builder Pattern - konstruksi objek kompleks bertahap
// 4. Singleton Pattern - satu instance global
// 5. Strategy Pattern - algoritma interchangeable

package main

import "fmt"

// =====
// 1. REPOSITORY PATTERN
// Abstraksi layer database, memudahkan switching DB
// =====

// User adalah entity/model
type User struct {
    ID    int
    Name string
}

// UserRepository adalah kontrak/interface untuk operasi user
// Dengan interface, kita bisa ganti implementasi database kapan saja
type UserRepository interface {
    GetUser(id int) (*User, error)
    CreateUser(name string) (*User, error)
    DeleteUser(id int) error
}

// InMemoryUserRepo adalah implementasi in-memory (untuk development)
type InMemoryUserRepo struct {
    users map[int]*User
    nextID int
}

func NewInMemoryUserRepo() *InMemoryUserRepo {
    return &InMemoryUserRepo{
        users:  make(map[int]*User),
        nextID: 1,
    }
}

func (r *InMemoryUserRepo) GetUser(id int) (*User, error) {
    if user, ok := r.users[id]; ok {
        return user, nil
    }
    return nil, fmt.Errorf("user %d tidak ditemukan", id)
}

func (r *InMemoryUserRepo) CreateUser(name string) (*User, error) {
    user := &User{ID: r.nextID, Name: name}
    r.users[r.nextID] = user
    r.nextID++
    return user, nil
}

func (r *InMemoryUserRepo) DeleteUser(id int) error {
    if _, ok := r.users[id]; !ok {
        return fmt.Errorf("user %d tidak ditemukan", id)
    }
    delete(r.users, id)
    return nil
}

// UserRepositoryPostgres adalah implementasi PostgreSQL (simulasi)
type UserRepositoryPostgres struct {
    // db *sql.DB // koneksi database
}

func (r *UserRepositoryPostgres) GetUser(id int) (*User, error) {
    // return db.QueryRow("SELECT ...", id)
    panic("implementasi postgres")
}
```

```

}
func (r *UserRepositoryPostgres) CreateUser(name string) (*User, error) {
    panic("implementasi postgres")
}
func (r *UserRepositoryPostgres) DeleteUser(id int) error {
    panic("implementasi postgres")
}

// UserService menggunakan UserRepository (dependency injection)
type UserService struct {
    repo UserRepository // tergantung interface, bukan implementasi
}

func NewUserService(repo UserRepository) *UserService {
    return &UserService{repo: repo}
}

func (s *UserService) Register(name string) (*User, error) {
    return s.repo.CreateUser(name)
}

// =====
// 2. FACTORY PATTERN
// Membuat objek berdasarkan tipe yang diminta
// =====

// Logger interface
type Logger interface {
    Log(msg string)
}

// ConsoleLogger mencetak ke console
type ConsoleLogger struct{}
func (c ConsoleLogger) Log(msg string) {
    fmt.Println("CONSOLE:", msg)
}

// FileLogger menulis ke file (simulasi)
type FileLogger struct{}
func (f FileLogger) Log(msg string) {
    // f.write("[FILE]" + msg)
    fmt.Println("FILE:", msg)
}

// NewLogger adalah factory function
func NewLogger(loggerType string) Logger {
    switch loggerType {
    case "file":
        return FileLogger{}
    default:
        return ConsoleLogger{}
    }
}

// =====
// 3. BUILDER PATTERN
// Membangun objek kompleks step-by-step
// =====

type Pizza struct {
    Size      string
    Crust     string
    Cheese    bool
    Pepperoni bool
    Mushrooms bool
    Olives    bool
}

type PizzaBuilder struct {
    pizza Pizza
}

func (b *PizzaBuilder) SetSize(size string) *PizzaBuilder {
    b.pizza.Size = size
    return b // return builder untuk method chaining
}

func (b *PizzaBuilder) SetCrust(crust string) *PizzaBuilder {
    b.pizza.Crust = crust
    return b
}

func (b *PizzaBuilder) AddCheese() *PizzaBuilder {
    b.pizza.Cheese = true
    return b
}

func (b *PizzaBuilder) AddPepperoni() *PizzaBuilder {
    b.pizza.Pepperoni = true
    return b
}

func (b *PizzaBuilder) Build() Pizza {
    return b.pizza
}

// =====
// 4. SINGLETON PATTERN
// Satu instance global untuk suatu objek
// =====

// singleton menggunakan sync.Once untuk thread safety
type AppConfig struct {
    Version string
    Env     string
}

var instance *AppConfig

```

```

func GetAppConfig() *AppConfig {
    // Initialize only once (sederhana, tanpa sync.Once)
    if instance == nil {
        instance = &AppConfig{
            Version: "1.0.0",
            Env:      "production",
        }
    }
    return instance
}

// =====
// MAIN
// =====

func main() {
    fmt.Println("=== Repository Pattern ===")
    repo := NewInMemoryUserRepo()
    service := NewUserService(repo)

    user, _ := service.Register("Anggi")
    fmt.Printf("Created: %v\n", user)
    user, _ = service.Register("Budi")
    fmt.Printf("Created: %v\n", user)
    fetched, _ := repo.GetUser(1)
    fmt.Printf("Fetched: %v\n", fetched)

    fmt.Println("\n=== Factory Pattern ===")
    consoleLogger := NewLogger("console")
    consoleLogger.Log("test message")
    fileLogger := NewLogger("file")
    fileLogger.Log("test message")

    fmt.Println("\n=== Builder Pattern ===")
    pizza := (&PizzaBuilder{}).
        SetSize("Large").
        SetCrust("Thin").
        AddCheese().
        AddPepperoni().
        Build()
    fmt.Printf("Pizza: size=%s, crust=%s, cheese=%t, pepperoni=%t\n",
        pizza.Size, pizza.Crust, pizza.Cheese, pizza.Pepperoni)

    fmt.Println("\n=== Singleton Pattern ===")
    cfg1 := GetAppConfig()
    cfg2 := GetAppConfig()
    fmt.Printf("Config: %v\n", cfg1)
    fmt.Println("Same instance:", cfg1 == cfg2)
}

```

Topik: websocket

```
// File: expert/websocket/main.go
// Level: Expert
// Topik: WebSocket Server
//
// WebSocket memungkinkan komunikasi real-time bidirectional
// antara client (browser) dan server.
//
// Cocok untuk:
// - Chat application
// - Live notifications
// - Real-time data (sports, stocks)
// - Collaborative editing
//
// Library: github.com/gorilla/websocket

package main

import (
    "fmt"
    "log"
    "net/http"
    "time"

    "github.com/gorilla/websocket"
)

// upgrader mengubah HTTP connection menjadi WebSocket connection
var upgrader = websocket.Upgrader{
    // CheckOrigin: izinkan koneksi dari origin manapun (development)
    CheckOrigin: func(r *http.Request) bool { return true },
    // Di production, batasi origin:
    // CheckOrigin: func(r *http.Request) bool {
    //     return r.Header.Get("Origin") == "https://app.example.com"
    // },
}

// handleWS adalah handler untuk WebSocket connections
func handleWS(w http.ResponseWriter, r *http.Request) {
    // Upgrade HTTP ke WebSocket
    conn, err := upgrader.Upgrade(w, r, nil)
    if err != nil {
        log.Println("Upgrade error:", err)
        return
    }
    defer conn.Close() // tutup koneksi saat selesai

    log.Printf("Client connected: %s", r.RemoteAddr)

    // Loop membaca pesan dari client
    for {
        // ReadMessage: blocking sampai ada pesan
        messageType, msg, err := conn.ReadMessage()
        if err != nil {
            // Client disconnect atau error
            if websocket.IsUnexpectedCloseError(err,
                websocket.CloseGoingAway, websocket.CloseNormalClosure) {
                log.Printf("Error: %v", err)
            }
            break
        }

        // Log pesan yang diterima
        log.Printf("Received: %s", msg)

        // Kirim response ke client
        response := fmt.Sprintf("Server menerima pada %s: %s",
            time.Now().Format(time.RFC3339), msg)

        err = conn.WriteMessage(messageType, []byte(response))
        if err != nil {
            log.Println("Write error:", err)
            break
        }
    }
}

func main() {
    // WebSocket endpoint
    http.HandleFunc("/ws", handleWS)

    // Serve HTML client
    http.HandleFunc("/", func(w http.ResponseWriter, r *http.Request) {
        http.ServeFile(w, r, "index.html")
    })

    log.Println("WebSocket server running on :8080")
    log.Println("Buka http://localhost:8080 di browser")
    log.Fatal(http.ListenAndServe(":8080", nil))
}

// Cara install:
// cd expert/websocket
// go get github.com/gorilla/websocket
// go run main.go
```

```

<!DOCTYPE html>
<html>
<head>
  <title>WebSocket Test</title>
</head>
<body>
  <h1>WebSocket Test</h1>
  <input id="msg" placeholder="Type message">
  <button onclick="send()">Send</button>
  <div id="output"></div>

  <script>
    const ws = new WebSocket("ws://localhost:8080/ws");
    ws.onmessage = (e) => {
      document.getElementById("output").innerHTML += `<p>Received: ${e.data}</p>`;
    };
    function send() {
      ws.send(document.getElementById("msg").value);
    }
  </script>
</body>
</html>

```

Topik: graphql

```

// File: expert/graphql/main.go
// Level: Expert
// Topik: GraphQL API Server
//
// GraphQL adalah query language untuk API yang dikembangkan oleh Facebook.
// Kelebihan dibanding REST: client bisa meminta field spesifik yang dibutuhkan.
//
// Library: github.com/graphql-go/graphql
//
// Cara install:
// cd expert/graphql
// go get github.com/graphql-go/graphql
// go run main.go

```

```
package main
```

```
import (
    "encoding/json"
    "fmt"
    "log"
    "net/http"

    "github.com/graphql-go/graphql"
)
```

```
// ===== TYPE DEFINITIONS =====
```

```
// Todo adalah model data
type Todo struct {
    ID      int    `json:"id"`
    Title   string `json:"title"`
    Done    bool   `json:"done"`
}
```

```
// Data dummy (biasanya dari database)
var todos = []Todo{
    {ID: 1, Title: "Belajar Go", Done: true},
    {ID: 2, Title: "Buat REST API", Done: false},
    {ID: 3, Title: "Belajar GraphQL", Done: false},
}
```

```
// ===== GRAPHQL SCHEMA DEFINITION =====
```

```
// todoType adalah representasi GraphQL dari struct Todo
var todoType = graphql.NewObject(graphql.ObjectConfig{
    Name: "Todo",
    Description: "Todo item dengan ID, title, dan status done",
    Fields: graphql.Fields{
        "id": &graphql.Field{
            Type: graphql.Int,
            Description: "ID unik todo",
        },
        "title": &graphql.Field{
            Type: graphql.String,
            Description: "Judul todo",
        },
        "done": &graphql.Field{
            Type: graphql.Boolean,
            Description: "Status selesai atau belum",
        },
    },
})
```

```
// queryType adalah entry point untuk query
var queryType = graphql.NewObject(graphql.ObjectConfig{
    Name: "Query",
    Fields: graphql.Fields{
        // Query: todo(id: 1) { id title done }
        "todo": &graphql.Field{
            Type: todoType,
            Description: "Mendapatkan todo berdasarkan ID",
            Args: graphql.FieldConfigArgument{
                "id": &graphql.ArgumentConfig{
                    Type: graphql.Int,
                    Description: "ID dari todo",
                },
            },
        },
    },
})
```

```

    },
    Resolve: func(p graphql.ResolveParams) (interface{}, error) {
        // Ambil argumen id
        id, _ := p.Args["id"].(int)
        // Cari di data
        for _, todo := range todos {
            if todo.ID == id {
                return todo, nil
            }
        }
        return nil, nil
    },
},
// Query: todos { id title done }
"todos": &graphql.Field{
    Type:      graphql.NewList(todoType),
    Description: "Mendapatkan semua todos",
    Resolve: func(p graphql.ResolveParams) (interface{}, error) {
        return todos, nil
    },
},
},
})

// Schema adalah GraphQL schema yang sudah dikompilasi
var schema, _ = graphql.NewSchema(graphql.SchemaConfig{
    Query: queryType,
})

// ===== HTTP HANDLER =====

// graphqlHandler menangani semua request GraphQL
func graphqlHandler(w http.ResponseWriter, r *http.Request) {
    // Parse request body
    var params struct {
        Query string `json:"query"`
    }
    err := json.NewDecoder(r.Body).Decode(&params)
    if err != nil {
        http.Error(w, "Invalid request body", http.StatusBadRequest)
        return
    }

    // Eksekusi query GraphQL
    result := graphql.Do(graphql.Params{
        Schema:      schema,
        RequestString: params.Query,
    })

    // Set response headers
    w.Header().Set("Content-Type", "application/json")
    w.Header().Set("Access-Control-Allow-Origin", "*")

    // Kirim response
    json.NewEncoder(w).Encode(result)
}

func main() {
    http.HandleFunc("/graphql", graphqlHandler)

    log.Println("GraphQL server running on :8080")
    log.Println("Test:")
    log.Println(`curl -X POST -d '{"query":{" todos { id title done } }}' localhost:8080/graphql`)
    log.Println(`curl -X POST -d '{"query":{" todo(id: 1) { id title done } }}' localhost:8080/graphql`)
    log.Fatal(http.ListenAndServe(":8080", nil))
}

```


Bab 4 — Fullstack Project

Proyek akhir: REST API Go (CRUD + JSON file storage) dengan frontend React (CDN/Vite).

Backend (api/)

fullstack/api/db_main.go

```
package main

import (
    "database/sql"
    "encoding/json"
    "log"
    "net/http"
    "strconv"

    _ "github.com/mattn/go-sqlite3"
)

type Todo struct {
    ID      int    `json:"id"`
    Title   string `json:"title"`
    Done    bool   `json:"done"`
}

var db *sql.DB

func initDB() {
    var err error
    db, err = sql.Open("sqlite3", "./todos.db")
    if err != nil {
        log.Fatal(err)
    }

    db.Exec(`CREATE TABLE IF NOT EXISTS todos (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        title TEXT NOT NULL,
        done INTEGER DEFAULT 0
    )`)
}

func handleTodos(w http.ResponseWriter, r *http.Request) {
    w.Header().Set("Content-Type", "application/json")
    w.Header().Set("Access-Control-Allow-Origin", "")

    switch r.Method {
    case http.MethodGet:
        rows, _ := db.Query("SELECT id, title, done FROM todos")
        defer rows.Close()
        var todos []Todo
        for rows.Next() {
            var t Todo
            var doneInt int
            rows.Scan(&t.ID, &t.Title, &doneInt)
            t.Done = doneInt == 1
            todos = append(todos, t)
        }
        json.NewEncoder(w).Encode(todos)

    case http.MethodPost:
        var todo Todo
        json.NewDecoder(r.Body).Decode(&todo)
        result, _ := db.Exec("INSERT INTO todos (title, done) VALUES (?, ?)", todo.Title, todo.Done)
        id, _ := result.LastInsertId()
        todo.ID = int(id)
        json.NewEncoder(w).Encode(todo)
    }
}

func handleTodo(w http.ResponseWriter, r *http.Request) {
    w.Header().Set("Content-Type", "application/json")
    w.Header().Set("Access-Control-Allow-Origin", "")

    id, _ := strconv.Atoi(r.URL.Query().Get("id"))

    switch r.Method {
    case http.MethodPut:
        var todo Todo
        json.NewDecoder(r.Body).Decode(&todo)
        db.Exec("UPDATE todos SET title = ?, done = ? WHERE id = ?", todo.Title, todo.Done, id)
        todo.ID = id
        json.NewEncoder(w).Encode(todo)

    case http.MethodDelete:
        db.Exec("DELETE FROM todos WHERE id = ?", id)
        w.WriteHeader(http.StatusNoContent)
    }
}

func main() {
    initDB()
    http.HandleFunc("/todos", handleTodos)
    http.HandleFunc("/todo", handleTodo)
    log.Println("Server running on :8080")
    log.Fatal(http.ListenAndServe(":8080", nil))
}
```

fullstack/api/json_main.go

```

package main

import (
    "encoding/json"
    "log"
    "net/http"
    "os"
    "strconv"
    "sync"
)

type Todo struct {
    ID      int    `json:"id"`
    Title   string `json:"title"`
    Done    bool   `json:"done"`
}

var (
    todos []Todo
    mu     sync.Mutex
    nextID = 1
    dataFile = "todos.json"
)

func loadData() {
    file, err := os.Open(dataFile)
    if err != nil {
        return
    }
    defer file.Close()
    json.NewDecoder(file).Decode(&todos)
    for _, t := range todos {
        if t.ID >= nextID {
            nextID = t.ID + 1
        }
    }
}

func saveData() {
    file, _ := os.Create(dataFile)
    defer file.Close()
    json.NewEncoder(file).Encode(todos)
}

func handleTodos(w http.ResponseWriter, r *http.Request) {
    w.Header().Set("Content-Type", "application/json")
    w.Header().Set("Access-Control-Allow-Origin", "*")

    switch r.Method {
    case http.MethodGet:
        mu.Lock()
        defer mu.Unlock()
        json.NewEncoder(w).Encode(todos)

    case http.MethodPost:
        var todo Todo
        json.NewDecoder(r.Body).Decode(&todo)
        mu.Lock()
        todo.ID = nextID
        nextID++
        todos = append(todos, todo)
        mu.Unlock()
        saveData()
        json.NewEncoder(w).Encode(todo)

    default:
        http.Error(w, "Method not allowed", http.StatusMethodNotAllowed)
    }
}

func handleTodo(w http.ResponseWriter, r *http.Request) {
    w.Header().Set("Content-Type", "application/json")
    w.Header().Set("Access-Control-Allow-Origin", "*")

    id, err := strconv.Atoi(r.URL.Query().Get("id"))
    if err != nil {
        http.Error(w, "Invalid ID", http.StatusBadRequest)
        return
    }

    mu.Lock()
    defer mu.Unlock()

    for i, todo := range todos {
        if todo.ID == id {
            switch r.Method {
            case http.MethodPut:
                var updated Todo
                json.NewDecoder(r.Body).Decode(&updated)
                todos[i].Title = updated.Title
                todos[i].Done = updated.Done
                saveData()
                json.NewEncoder(w).Encode(todos[i])

            case http.MethodDelete:
                todos = append(todos[:i], todos[i+1:]...)
                saveData()
                w.WriteHeader(http.StatusNoContent)

            default:
                http.Error(w, "Method not allowed", http.StatusMethodNotAllowed)
            }
            return
        }
    }
    http.Error(w, "Todo not found", http.StatusNotFound)
}

```

```
func main() {
    loadData()
    http.HandleFunc("/todos", handleTodos)
    http.HandleFunc("/todo", handleTodo)
    log.Println("Server running on :8080")
    log.Fatal(http.ListenAndServe(":8080", nil))
}
```

📄 fullstack/api/main.go

```
// File: fullstack/api/main.go
// Level: Middle (Fullstack)
// Topik: CRUD REST API dengan JSON File Storage
//
// REST API untuk Todo CRUD dengan data disimpan di JSON file.
// Fitur: GET, POST, PUT, DELETE dengan persistensi ke file.
// CORS enabled untuk frontend React.
//
// Endpoints:
// GET /todos -> List semua todos
// POST /todos -> Tambah todo baru (body: {"title":"...", "done":false})
// PUT /todo?id=1 -> Update todo (body: {"title":"...", "done":true})
// DELETE /todo?id=1 -> Hapus todo
```

```
package main
```

```
import (
    "encoding/json" // untuk encode/decode JSON
    "log"           // untuk logging
    "net/http"      // HTTP server
    "os"            // file I/O
    "strconv"       // konversi string ke int
    "sync"          // mutual exclusion untuk concurrent access
)
```

```
// Todo merepresentasikan item todo
```

```
type Todo struct {
    ID      int    `json:"id"` // ID unik
    Title   string `json:"title"` // Judul todo
    Done    bool   `json:"done"` // Status selesai/belum
}
```

```
// Global state
```

```
var (
    todos []Todo // slice untuk menyimpan todos di memori
    mu     sync.Mutex // mutex untuk thread safety
    nextID = 1 // auto-increment ID
    dataFile = "todos.json" // file untuk persistensi data
)
```

```
// loadData membaca data dari JSON file ke memory
```

```
func loadData() {
    file, err := os.Open(dataFile)
    if err != nil {
        // File belum ada (pertama kali jalan)
        return
    }
    defer file.Close()

    json.NewDecoder(file).Decode(&todos)

    // Set nextID berdasarkan ID terakhir di file
    for _, t := range todos {
        if t.ID >= nextID {
            nextID = t.ID + 1
        }
    }
}
```

```
// saveData menulis data dari memory ke JSON file
```

```
func saveData() {
    file, err := os.Create(dataFile)
    if err != nil {
        log.Printf("Error saving data: %v", err)
        return
    }
    defer file.Close()

    encoder := json.NewEncoder(file)
    encoder.SetIndent("", " ") // pretty print JSON
    encoder.Encode(todos)
}
```

```
// handleTodos menangani GET /todos dan POST /todos
```

```
func handleTodos(w http.ResponseWriter, r *http.Request) {
    // Set response headers
    w.Header().Set("Content-Type", "application/json")
    w.Header().Set("Access-Control-Allow-Origin", "*") // CORS
    w.Header().Set("Access-Control-Allow-Methods", "GET, POST, OPTIONS")
    w.Header().Set("Access-Control-Allow-Headers", "Content-Type")

    // Handle CORS preflight
    if r.Method == http.MethodOptions {
        w.WriteHeader(http.StatusOK)
        return
    }
}
```

```
// Routing berdasarkan HTTP method
```

```
switch r.Method {
case http.MethodGet:
    // GET /todos - Kembalikan semua todos
    mu.Lock()
    defer mu.Unlock()
    json.NewEncoder(w).Encode(todos)

case http.MethodPost:
    // POST /todos - Tambah todo baru
```

```

    var todo Todo
    err := json.NewDecoder(r.Body).Decode(&todo)
    if err != nil {
        http.Error(w, "Invalid JSON body", http.StatusBadRequest)
        return
    }

    mu.Lock()
    todo.ID = nextID // generate ID baru
    nextID++
    todos = append(todos, todo)
    mu.Unlock()

    saveData() // persist ke file

    w.WriteHeader(http.StatusCreated) // HTTP 201
    json.NewEncoder(w).Encode(todo)

default:
    http.Error(w, "Method not allowed", http.StatusMethodNotAllowed)
}
}

// handleTodo menangani PUT /todo?id=1 dan DELETE /todo?id=1
func handleTodo(w http.ResponseWriter, r *http.Request) {
    // Set response headers
    w.Header().Set("Content-Type", "application/json")
    w.Header().Set("Access-Control-Allow-Origin", "*")
    w.Header().Set("Access-Control-Allow-Methods", "PUT, DELETE, OPTIONS")
    w.Header().Set("Access-Control-Allow-Headers", "Content-Type")

    if r.Method == http.MethodOptions {
        w.WriteHeader(http.StatusOK)
        return
    }

    // Parse ID dari query parameter
    idStr := r.URL.Query().Get("id")
    id, err := strconv.Atoi(idStr)
    if err != nil {
        http.Error(w, `{"error": "Invalid ID"}`, http.StatusBadRequest)
        return
    }

    mu.Lock()
defer mu.Unlock()

    // Cari todo dengan ID yang sesuai
    for i, todo := range todos {
        if todo.ID == id {
            switch r.Method {
            case http.MethodPut:
                // PUT /todo?id=1 - Update todo
                var updated Todo
                err := json.NewDecoder(r.Body).Decode(&updated)
                if err != nil {
                    http.Error(w, "Invalid JSON body", http.StatusBadRequest)
                    return
                }
                todos[i].Title = updated.Title
                todos[i].Done = updated.Done
                saveData()
                json.NewEncoder(w).Encode(todos[i])

            case http.MethodDelete:
                // DELETE /todo?id=1 - Hapus todo
                todos = append(todos[:i], todos[i+1:]...)
                saveData()
                w.WriteHeader(http.StatusNoContent) // HTTP 204

            default:
                http.Error(w, "Method not allowed", http.StatusMethodNotAllowed)
            }
            return // todo ditemukan, selesai
        }
    }

    // Todo tidak ditemukan
    http.Error(w, `{"error": "Todo not found"}`, http.StatusNotFound)
}

func main() {
    // Load data dari file saat startup
    loadData()
    log.Printf("Loaded %d todos from %s", len(todos), dataFile)

    // Register routes
    http.HandleFunc("/todos", handleTodos) // collection route
    http.HandleFunc("/todo", handleTodo)   // single item route

    // Start server
    log.Println("Server running on http://localhost:8080")
    log.Fatal(http.ListenAndServe(":8080", nil))
}

/*
Cara menjalankan:
cd fullstack/api
go run main.go

Coba endpoints:
# List semua todos
curl localhost:8080/todos

# Tambah todo
curl -X POST -H "Content-Type: application/json" \
-d '{"title": "Belajar Go", "done": false}' \
localhost:8080/todos

```

```
# Update todo
curl -X PUT -H "Content-Type: application/json" \
-d '{"title":"Belajar Go","done":true}' \
"localhost:8080/todo?id=1"

# Hapus todo
curl -X DELETE "localhost:8080/todo?id=1"
*/
```

Frontend (frontend/)

fullstack/frontend/index.html

```
<!--
File: fullstack/frontend/index.html
Level: Middle (Fullstack)
Topik: React Frontend dengan CDN

Frontend React sederhana untuk Todo CRUD.
Menggunakan React via CDN (tanpa bundler/webpack).
Berkomunikasi dengan Go API di localhost:8080.

Fitur:
- List semua todos
- Tambah todo baru
- Toggle done/undone
- Hapus todo

Cara menjalankan:
1. Jalankan backend: cd fullstack/api && go run main.go
2. Buka file ini di browser (double click)
3. Atau: python3 -m http.server 3000
-->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Go CRUD Todo</title>

  <!-- React via CDN -->
  <script src="https://unpkg.com/react@18/umd/react.development.js"></script>
  <script src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"></script>
  <!-- Babel untuk JSX transpilation -->
  <script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>

  <style>
    body {
      font-family: Arial, sans-serif;
      max-width: 500px;
      margin: 50px auto;
      padding: 20px;
      background: #f5f5f5;
    }
    h1 { color: #333; }
    .todo-item {
      padding: 10px;
      background: white;
      margin: 5px 0;
      border-radius: 4px;
      display: flex;
      justify-content: space-between;
      align-items: center;
      box-shadow: 0 1px 3px rgba(0,0,0,0.1);
    }
    .done {
      text-decoration: line-through;
      color: #888;
    }
    input {
      padding: 8px;
      flex: 1;
      border: 1px solid #ddd;
      border-radius: 4px;
    }
    button {
      background: #007bff;
      color: white;
      border: none;
      padding: 8px 15px;
      margin-left: 5px;
      border-radius: 4px;
      cursor: pointer;
    }
    button:hover { background: #0056b3; }
    button.delete { background: #dc3545; }
    button.delete:hover { background: #c82333; }
    .input-group { display: flex; margin-bottom: 20px; }
  </style>
</head>
<body>
  <div id="root"></div>

  <!-- React component dengan JSX -->
  <script type="text/babel">
    const { useState, useEffect } = React;

    // API base URL - sesuaikan dengan backend Go
    const API_URL = 'http://localhost:8080';

    // Komponen utama App
    function App() {
      // State: todos dan title input
      const [todos, setTodos] = useState([]);
      const [title, setTitle] = useState('');
    }
  </script>
</body>
</html>
```

```

const [loading, setLoading] = useState(true);

// useEffect: mengambil data dari API saat komponen dimuat
useEffect(() => {
  fetchTodos();
}, []);

// Fungsi untuk mengambil semua todos dari API
const fetchTodos = () => {
  setLoading(true);
  fetch(`${API_URL}/todos`)
    .then(res => res.json())
    .then(data => {
      setTodos(data);
      setLoading(false);
    })
    .catch(err => {
      console.error('Error fetching todos:', err);
      setLoading(false);
    });
};

// Fungsi untuk menambah todo baru
const addTodo = () => {
  if (!title.trim()) return;

  fetch(`${API_URL}/todos`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ title, done: false })
  })
    .then(res => res.json())
    .then(newTodo => {
      setTodos([...todos, newTodo]); // tambah ke state
      setTitle(''); // reset input
    })
    .catch(err => console.error('Error adding todo:', err));
};

// Fungsi untuk toggle status done/undone
const toggleTodo = (id, currentDone) => {
  const todo = todos.find(t => t.id === id);

  fetch(`${API_URL}/todo?id=${id}`, {
    method: 'PUT',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ ...todo, done: !currentDone })
  })
    .then(res => res.json())
    .then(updated => {
      setTodos(todos.map(t => t.id === id ? updated : t));
    })
    .catch(err => console.error('Error updating todo:', err));
};

// Fungsi untuk menghapus todo
const deleteTodo = (id) => {
  fetch(`${API_URL}/todo?id=${id}`, { method: 'DELETE' })
    .then(() => {
      setTodos(todos.filter(t => t.id !== id));
    })
    .catch(err => console.error('Error deleting todo:', err));
};

// Render komponen
return (
  <div>
    <h1>Todo List <small style={{fontSize:'14px',color:'#888'}}>(Go API)</small></h1>

    {/* Input form */}
    <div className="input-group">
      <input
        value={title}
        onChange={e => setTitle(e.target.value)}
        onKeyDown={e => e.key === 'Enter' && addTodo()}
        placeholder="Tambah todo baru..."
      />
      <button onClick={addTodo}>Add</button>
    </div>

    {/* Loading indicator */}
    {loading && <p>Loading...</p>}

    {/* List todos */}
    {!loading && todos.length === 0 && <p>Belum ada todo. Tambahkan!</p>}

    {todos.map(todo => (
      <div key={todo.id} className="todo-item">
        <span className={todo.done ? 'done' : ''}>
          {todo.title}
        </span>
        <div>
          <button onClick={() => toggleTodo(todo.id, todo.done)}>
            {todo.done ? 'Undo' : 'Done'}
          </button>
          <button className="delete" onClick={() => deleteTodo(todo.id)}>
            X
          </button>
        </div>
      </div>
    )))}
  </div>
);
}

// Render React app ke DOM
ReactDOM.createRoot(document.getElementById('root')).render(<App />);
</script>
</body>

```

</html>

📄 fullstack/frontend/package.json

```
{
  "name": "go-crud-frontend",
  "version": "1.0.0",
  "description": "React frontend for Go CRUD API",
  "scripts": {
    "start": "vite",
    "build": "vite build"
  },
  "dependencies": {
    "react": "^18.2.0",
    "react-dom": "^18.2.0"
  },
  "devDependencies": {
    "vite": "^5.0.0"
  }
}
```

Konfigurasi

📄 fullstack/package.json

```
{
  "name": "go-todo-api",
  "version": "1.0.0",
  "description": "REST API for Todo CRUD",
  "main": "api/main.go",
  "scripts": {
    "start": "go run api/main.go",
    "test": "go test ./... -v"
  }
}
```